

Готовим мультисимвольные мультипериодные индикаторы

26 Artyom Trishkin 3 138

Содержание

- Введение
- Основные принципы
- Базовый класс MSTF-индикатора
- Классы индикаторов по типам
- Класс-коллекция индикаторов
- Тестирование
- Заключение

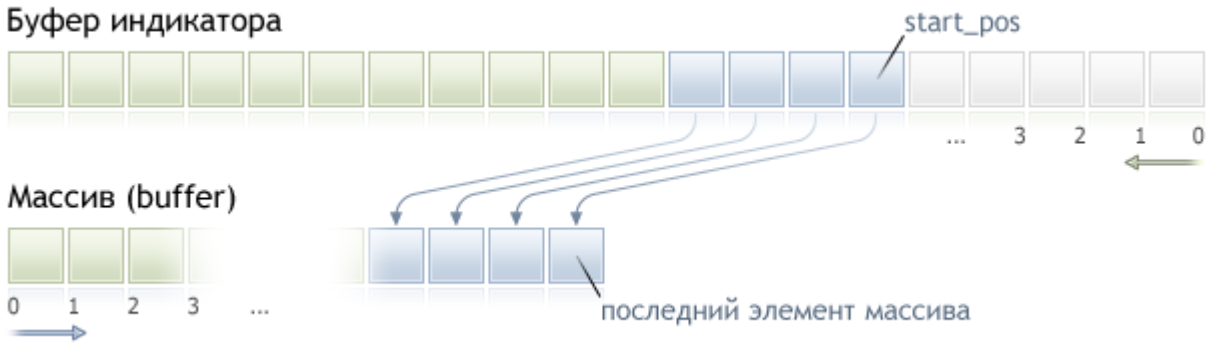
Введение

В продолжение темы создания шаблонов технических индикаторов в советниках и индикаторах, начатой в статье о создании информационной панели, и развивающейся на протяжении ещё трёх статей об осцилляторах, индикаторах объёма и Билла Вильямса, а так же трендовых индикаторах, сегодня начнём тему о создании мультисимвольных мультипериодных индикаторов. Это индикаторы, работающие на текущем графике, но рассчитанные на данных других символов и (или) иных таймфреймов графиков. Создадим один базовый класс мульти-индикатора и набор классов по типам всех стандартных индикаторов. Для пользовательского индикатора тоже будет создан класс, что даст нам возможность "превратить" любой индикатор в мультисимвольный и мультипериодный. Для всех классов индикаторов создадим один общий класс-коллекцию, в которую будут размещаться создаваемые в программе индикаторы, и можно будет методами коллекции обращаться к любому из созданных индикаторов для получения его данных. В общем, цель будет такой: сделать удобное средство для создания мульти-индикаторов и работы с их данными.

Основные принципы

Для правильного понимания логики работы с индикаторами, попробуем немного разобраться как это всё обустроено. У индикатора есть две части: расчётная и рисуемая. Каждая из этих частей ничего не знает про другую. При создании индикатора, подсистема терминала смотрит наличие такого индикатора на графике — с теми же именем и параметрами. Если такой индикатор уже прикреплён к графику, или создан для него программно, то используется хэндл уже имеющегося индикатора, а не создаётся новый. Рисуемая часть индикатора получает данные для отрисовки от расчётной части по её хэндлу. Может быть такая ситуация, что к одной расчётной части обращаются сразу несколько рисуемых.

В буфере расчётной части данные рассчитанного индикатора хранятся в массиве с расположением данных от настоящего к прошлому. Данные в индексе 0 массива-буфера соответствуют текущим данным графика:



В каждой ячейке массива хранятся данные одного бара, соответствующие бару таймсерии того символа/периода, на которых рассчитан индикатор. Соответственно, чтобы получить данные из буфера расчётной части индикатора и отобразить их на графике другого символа/таймфрейма, нужно рассчитать номер бара на графике, соответствующий времени бара в массиве-буфере расчётной части. Полученные данные записать в буфер рисуемой части индикатора так, чтобы все бары текущего графика, попадающие на время открытия бара в буфере расчётной части были вписаны в соответствующие ячейки рисуемого буфера.

Например, одному бару на пятиминутном периоде графика соответствуют пять баров минутного графика. Все эти пять баров минутного графика должны быть заполнены значением соответствующего им по времени бара пятиминутного графика. Похожий алгоритм используется и для рисования данных с младших периодов графика на графике со старшим периодом. В этом случае все бары ячеек буфера расчётной части, соответствующие времени бара на графике старшего таймфрейма, рисуются на одном баре рисуемого буфера.

Здесь будет, естественно, некое допущение в показаниях, так как в итоге лишь данные последнего бара младшего таймфрейма, попадающие по времени на соответствующий бар старшего таймфрейма, будут отрисованы на этом баре. Тут всё зависит от направления получения данных из буфера расчётной части младшего таймфрейма — какие данные получены последними, те и будут нарисованы на баре графика старшего таймфрейма.

Для получения данных из буфера рассчитанного индикатора предназначена функция CopyBuffer():

> Получает в массив buffer данные указанного буфера указанного индикатора в указанном количестве.
>
> Отсчет элементов копируемых данных (индикаторный буфер с индексом buffer_num) от стартовой позиции ведется от настоящего к прошлому, то есть стартовая позиция, равная 0, означает текущий бар (значение индикатора для текущего бара).
>
> При копировании заранее неизвестного количества данных в качестве массива-приемника buffer[] желательно использовать динамический массив, так как функция CopyBuffer() старается распределить размер принимающего массива под размер копируемых данных. Если в качестве принимающего массива buffer[] выступает индикаторный буфер (массив, предварительно назначенный под хранение значений индикатора функцией SetIndexBufer()), то допускается частичное копирование.
>
> Если необходимо произвести частичное копирование значений индикатора в другой массив (не индикаторный буфер), то для этих целей необходимо использовать промежуточный массив, в который копируется требуемое количество. И уже из этого массива-посредника произвести поэлементное копирование нужного количества значений в нужные места принимающего массива.
>
> Если необходимо копировать заранее известное количество данных, то лучше это делать в статически выделенный буфер, чтобы избежать излишнего перевыделения памяти.

Неважно, какое свойство имеет приемный массив — as_series=true или as_series=false, данные будут скопированы таким образом, что самый старый по времени элемент будет в начале физической памяти, отведенной под массив. Существует 3 варианта функции.

Обращение по начальной позиции и количеству требуемых элементов

```
int  CopyBuffer(
   int       indicator_handle,     // handle индикатора
   int       buffer_num,           // номер буфера индикатора
   int       start_pos,            // откуда начнем
   int       count,                // сколько копируем
   double    buffer[]              // массив, куда будут скопированы данные
   );
```

Обращение по начальной дате и количеству требуемых элементов

```
int  CopyBuffer(
   int       indicator_handle,     // handle индикатора
   int       buffer_num,           // номер буфера индикатора
   datetime  start_time,           // с какой даты
   int       count,                // сколько копируем
   double    buffer[]              // массив, куда будут скопированы данные
   );
```

Обращение по начальной и конечной датам требуемого интервала времени

```
int  CopyBuffer(
   int       indicator_handle,     // handle индикатора
   int       buffer_num,           // номер буфера индикатора
   datetime  start_time,           // с какой даты
   datetime  stop_time,            // по какую дату
   double    buffer[]              // массив, куда будут скопированы данные
   );
```

Параметры

indicator_handle

[in] Хэндл индикатора, полученный соответствующей индикаторной функцией.

buffer_num

[in] Номер индикаторного буфера.

start_pos

[in] Номер первого копируемого элемента.

count

[in] Количество копируемых элементов.

start_time

[in] Время бара, соответствующее первому элементу.

stop_time

[in] Время бара, соответствующее последнему элементу.

buffer[]

[out] Массив типа double.

Возвращаемое значение

Количество скопированных элементов массива либо -1 в случае ошибки.

Примечание

При запросе данных из индикатора, если запрашиваемые таймсерии еще не построены или их необходимо загрузить с сервера, то функция сразу же вернет -1, но при этом сам процесс загрузки/построения будет инициирован.

При запросе данных из эксперта или скрипта, будет инициирована загрузка с сервера, если локально этих данных у терминала нет, либо начнется построение нужной таймсерии, если данные можно построить из локальной истории, но они еще не готовы. Функция вернет то количество данных, которые будут готовы к моменту истечения таймаута.

Мы будем использовать первый вариант функции — обращение по начальной позиции (индексу цикла) и количеству требуемых элементов.

Структура объектов будет такой:

1. Базовый класс мультисимвольного мультипериодного индикатора, содержащего основной, общий для всех индикаторов, функционал;
2. Производные от базового объекта классы, описывающие каждый индикатор по его типу;
3. Класс-коллекция индикаторов, позволяющий создавать любые индикаторы и добавлять их в коллекцию. Класс будет давать пользователю весь инструментарий для создания индикаторов и получения от них данных.

При работе с данными не текущего графика для исключения "освобождения" таймсерии, необходимо не реже чем раз в две минуты обращаться к этой таймсерии. В этом случае будет происходить "удержание" таймсерии, что ускорит к ней обращение (не нужно будет каждый раз дожидаться синхронизации данных). В конструкторе класса будем делать первое обращение к таймсерии, на которой строится индикатор, что позволит начать её загрузку (при отсутствии доступа к ней локально). И далее, раз в 90 секунд (полторы минуты) в таймере базового класса будем обращаться к таймсерии для её удержания.

Экономный расчёт индикатора

Для расчёта и отображения индикатора необходимо сделать экономный просчёт — чтобы при первом запуске индикатор мог рассчитаться на всех исторических данных, а на последующих тиках — только один-два бара.

В обработчике OnCalculate() есть предопределённые константные переменные, хранящие размер данных входных данных (таймсерии или массива) и количество рассчитанных данных на прошлом вызове OnCalculate():

Вычисление на основе массива данных

```
int  OnCalculate(
   const int        rates_total,       // размер массива price[]
   const int        prev_calculated,   // количество обработанных баров на предыд
   const int        begin,             // номер индекса в массиве price[], с кото
   const double&    price[]            // массив значений для расчета
   );
```

Вычисления на основе таймсерий текущего таймфрейма

```
int  OnCalculate(
   const int        rates_total,       // размер входных таймсерий
```

```
    const int        prev_calculated,   // количество обработанных баров на предыд
    const datetime&  time[],            // массив Time
    const double&    open[],            // массив Open
    const double&    high[],            // массив High
    const double&    low[],             // массив Low
    const double&    close[],           // массив Close
    const long&      tick_volume[],     // массив Tick Volume
    const long&      volume[],          // массив Real Volume
    const int&       spread[]           // массив Spread
    );
```

Наличие таких данных позволяет сделать быстрый экономный расчёт индикатора. Например, одним из возможных вариантов такого расчёта является подсчёт значения limit:

```
//--- Количество баров для расчёта
   int limit=rates_total-prev_calculated;
//--- Если limit > 1, значит это первый расчёт, либо изменение в истории
   if(limit>1)
     {
      //--- указываем для просчёта всю доступную историю
      limit=rates_total-1;
      //--- Если в индикаторе есть какие-либо буферы, то здесь их нужно инициализироват
     }
```

При первом запуске индикатора мы имеем размер таймсерии (rates_total) и количество рассчитанных данных на прошлом вызове (prev_calculated). Значение ранее рассчитанных баров при первом запуске равно нулю — индикатор ещё не рассчитывался. Таким образом значение limit будет больше 1 (будет равно количеству доступных баров минус ноль). При таком значении указываем величину limit равной rates_total-1 — это вся доступная история для расчёта. В этом случае дополнительно нужно сначала стереть все ранее нарисованные данные в буферах индикатора, например

```
ArrayInitialize(Buffer,EMPTY_VALUE);
```

После этого в основном цикле будет просчитана полностью вся история, так как цикл строится от значения limit до нуля включительно:

```
   for(int i=limit;i>=0;i--)
     {
      // Расчёт индикатора на каждом баре цикла (i)
     }
```

Следует отметить, что при таком способе расчёта все используемые в расчётах массивы и сам буфер индикатора должны иметь флаг индексации как у таймсерии:

```
ArraySetAsSeries(array,true);
```

Если рассчитанный limit равен 1, то это означает открытие нового бара на графике: индикатор посчитает первый и нулевой бары таймсерии — цикл от 1 до 0 включительно.

Если рассчитанный limit равен 0, то это означает работу на текущем тике: индикатор посчитает только нулевой бар таймсерии — цикл от 0 до 0 включительно.

Если нужен расчёт от нулевого бара вглубь исторических данных (чтобы не разворачивать массивы-таймсерии и буфера), то конструкция цикла будет обратной:

```
int i=fmax(0, prev_calculated-1);
for(;i<rates_total;i++)
  {
   // Расчёт индикатора на каждом баре цикла (i)
  }
```

Хорошо, экономный расчёт на текущем графике легко сделать. Но что делать, если нужны данные не с текущего графика? Когда копировать весь массив данных из расчётной части, а когда только один-две последних бара?

Вот тут на помощь приходят функции Bars() и BarsCalculated(). Это аналоги предопределённых константных переменных индикатора rates_total и prev_calculated, но возвращают количество баров для указанного символа/периода и количество просчитанных индикатором данных. Так как индикатор построен на символе/периоде, указываемых при его создании, то и количество просчитанных данных относится к этому символу/периоду. Индикатор же получаем по его хэндлу.

Исходя из того, что мы можем для любого символа/периода рассчитать сколько баров необходимо копировать (чтобы на каждом тике не копировать весь массив), в базовом классе индикатора мы сделаем точно такую же конструкцию, как и для текущего символа/периода:

```
limit=rates_total-prev_calculated;
```

только переменные limit, rates_total и prev_calculated будут приватными членами класса, а получать значения они будут из функций Bars() и BarsCalculated(). На каждом тике будет рассчитываться limit, и, если он нулевой, то копироваться будут только данные двух последних баров данных (текущий и предыдущий бары). Если limit равен 1, то это означает открытие нового бара на символе/периоде индикатора, и нужно увеличить массив на 1, а потом уже копировать данные из буфера индикатора — тоже всего два. При limit больше единицы копируется весь массив из буфера расчётной части индикатора в приёмный массив-буфер класса, так как считается, что это либо первый запуск, либо изменение в истории.

Такая логика относится к торговому дню — когда идут тики.

Выходной же день требует иного подхода. Это обособленный случай. Здесь тиков нет, Функция Bars() очень часто возвращает ноль, а количество просчитанных баров индикатора не фиксируется — тоже ноль. Индикатор, если есть какая-то ошибка в исходных для расчёта данных, должен вернуть ноль — чтобы до следующего тика подождать обновления данных и рассчитаться заново. Но в выходной день нет тика кроме первого запуска.

На первом запуске индикатор сначала очистит массивы буферов, затем рассчитается. Но расчёт может не получиться из-за недостаточности данных для расчёта, либо просто потому, что prev_calculated вернёт нулевое значение. И индикатор выйдет из OnCalculate(), вернув опять ноль. Соответственно, если обновить график ПКМ, что является по сути эмуляцией тика, то индикатор опять будет видеть, что он рассчитан с ошибкой, и опять инициализирует буферы, считая, что это первый запуск. И так по кругу, стирая только что нарисованные на графике данные рисуемых буферов... Но выход есть.

Если при первом запуске не получилось сразу рассчитать индикатор, то можно подождать следующего тика 20 — 30 секунд и эмулировать тик программно. Это можно сделать при помощи функции ChartSetSymbolPeriod(). Если её вызвать, указав текущие символ/период графика, то это вызовет перерасчет индикаторов, прикрепленных к графику. Таким образом, можно рассчитать индикатор на графике даже при отсутствии тиков.

Двадцатисекундного ожидания вполне достаточно для подгрузки нужной истории по символу/периоду индикатора и его расчёта. Но нам нужен флаг того, что индикатор уже посчитан (так как prev_calculated возвращает ноль), и если не поставить флаг успешности расчёта, то индикатор опять сотрёт свои буферы. Поэтому, для понимания, что индикатор успешно рассчитан, нам достаточно увидеть, что количество просчитанных баров у объекта-индикатора равно количеству баров на его символе/периоде.

Если же Bars() возвращает ноль, то мы можем другим способом узнать доступное количество баров нужного символа/периода (не забываем, что речь идёт о мультисимвольных мультипериодных индикаторах, которые рассчитываются в другой программе — советнике, или индикаторе, запущенных на текущем графике). В функции SeriesInfoInteger() мы можем получить дату начала доступной истории по символу/периоду и дату конца этой истории:

```
SeriesInfoInteger(symbol,period,SERIES_FIRSTDATE);       // Самая первая дата по симе
SeriesInfoInteger(symbol,period,SERIES_LASTBAR_DATE);    // Время открытия последнего ба
```

Из этих двух дат и периода графика таймсерии мы легко можем рассчитать количество доступных баров, даже, если Bars(symbol,period), либо SeriesInfoInteger(symbol,period,SERIES_BARS_COUNT) вернули ноль.

После того, как все данные будут получены и индикатор верно будет рассчитан, устанавливается флаг успешности расчёта. Такой флаг проверяется в условии, когда limit > 1 (в этом случае необходимо инициализировать массивы буферов индикатора "пустым" значением). При первом запуске флаг успешности сброшен — массивы инициализируются и делается попытка просчёта индикатора на не текущих символе/периоде графика. Если не получилось рассчитать, то ожидается 20 секунд в таймере следующего тика.

Если это выходной, то тика не будет, и через 20 секунд отправляется команда на установку графику текущих символа и периода — эмуляция тика. При перезапуске индикатора в таймере (спустя 20 секунд) данные уже должны быть подгружены и индикатор должен быть рассчитан без ошибок. При этом флаг успешности расчёта устанавливается. При следующем заходе в таймере проверяется этот флаг и, если он установлен, то тик не эмулируется. Всего таких попыток отрисовать буферы индикатора — три. По истечению трёх попыток принудительно устанавливается флаг успешности, и попытки эмуляции тика прекращаются. Если индикатор не смог рассчитаться за три эмулированных тика, значит теперь только вручную: либо (ПКМ --> Обновить), либо переключить таймфрейм графика туда-обратно, чтобы заново пройти весь процесс эмуляции тиков с подгрузкой данных.

Такова теория. Приступим к практике — созданию классов мульти-индикаторов.

Базовый класс MSTF-индикатора

В папке терминала \MQL5\Include\IndMSTF\ создадим новый файл IndMSTF.mqh класса CIndMSTF. Класс должен быть унаследован от базового объекта Стандартной Библиотеки CObject. Файл базового объекта должен быть подключен к созданному файлу нового класса:

```
//+------------------------------------------------------------------+
//|                                                    IndMSTF.mqh |
//|                                Copyright 2023, MetaQuotes Ltd. |
//|                                             https://www.mql5.com |
//+------------------------------------------------------------------+
#property copyright "Copyright 2023, MetaQuotes Ltd."
#property link      "https://www.mql5.com"
#property version   "1.00"
//--- includes
#include <Object.mqh>
//+------------------------------------------------------------------+
//| Базовый класс мультисимвольного мультипериодного индикатора      |
//+------------------------------------------------------------------+
class CIndMSTF : public CObject
  {
private:

protected:
```

```
public:
//--- Конструктор/деструктор
                     CIndMSTF();
                    ~CIndMSTF();
  };
```

Прежде, чем добавим методы в различные секции класса, добавим некоторые макроподстановки, перечисления и структуру буфера индикатора.

Нам нужен будет секундный таймер, в котором будут отслеживаться два промежутка времени:

- 90 секунд, по истечении которых будем обращаться к таймсериям индикаторов, рассчитанных не на текущих символе/периоде,
- 20 секунд, по истечении которых будем эмулировать тик для прорисовки индикатора в выходные дни.

Впишем макроподстановки для задания времени ожидания этих двух промежутков времени:

```
//--- includes
#include <Object.mqh>
//--- defines
#define TIMER_COUNT_1   (90)  // Размер таймера 1. Не должен быть более двух минут (120
#define TIMER_COUNT_2   (20)  // Размер таймера 2. Слишком малые значения быстро запуск
```

Разные стандартные индикаторы в клиентском терминале относятся к разным категориям. Для того, чтобы мы могли отсортировать созданные индикаторы по категориям или создавать списки индикаторов, относящиеся к какой-либо категории, напишем перечисление различных категорий индикаторов:

```
//--- defines
#define TIMER_COUNT_1   (90)  // Размер таймера 1. Не должен быть более двух минут (120
#define TIMER_COUNT_2   (20)  // Размер таймера 2. Слишком малые значения быстро запуск
//--- enums
enum ENUM_IND_CATEGORY         // Категории индикаторов
  {
   IND_CATEGORY_NONE,          // Не задана
   IND_CATEGORY_TREND,         // Трендовые
   IND_CATEGORY_OSCILLATOR,    // Осцилляторы
   IND_CATEGORY_VOLUME,        // Объемы
   IND_CATEGORY_WILLIAMS,      // Билла Вильямса
   IND_CATEGORY_CUSTOM,        // Пользовательские
  };
```

О сортировке, поиске и фильтрации

У каждого объекта индикатора будут свойства, по которым можно будет найти нужный индикатор. Чтобы понять, что индикаторы аналогичны друг другу, нужно сравнить их ключевые свойства — символ, период графика и значения всех входных параметров. Если хоть одно значение из сравниваемых свойств отличается, то индикаторы не идентичны. При равенстве индикаторов, новый создаваться не будет, а будет возвращаться указатель на ранее созданный с теми же параметрами. Это относительно коллекции индикаторов. Относительно же свойств, нам нужно создать перечисление, в котором будут прописаны константы некоторых свойств, которые можно будет задать успешно созданному индикатору, а потом по ним его найти:

```
enum ENUM_COMPARE_MODE          // Режим сравнения
  {
   // По умолчанию режим сравнения имеет нулевое значение, что производит сравнение по
   COMPARE_MODE_HANDLE=1,       // Сравнение по хэндлу
   COMPARE_MODE_SYMBOL,         // Сравнение по символу
   COMPARE_MODE_TIMEFRAME,      // Сравнение по периоду графика
   COMPARE_MODE_ID,             // Сравнение по идентификатору
   COMPARE_MODE_DESCRIPTION,    // Сравнение по пользовательскому описанию
   COMPARE_MODE_CATEGORY,       // Сравнение по категории
  };
```

Хэндл есть у каждого успешно созданного индикатора, и по этому хэндлу мы можем обращаться к нему. Это уникальный номер, присваиваемый расчётной части индикатора. Значения хэндлов создаваемых индикаторов начинаются с 10 и для каждого последующего значение увеличивается на 1.

Остальные свойства не являются уникальными, и могут быть присущи разным индикаторам. Поиск по ним здесь лишь для удобства. Например, можно задать уникальное описание для индикатора, а затем обращаться к нему по этому описанию.

Описания состояний линий индикатора мы уже рассматривали ранее в предыдущих статьях по индикаторам. Здесь тоже будем использовать это перечисление:

```
enum ENUM_LINE_STATE            // Состояние линии индикатора
  {
   LINE_STATE_NONE,             // Неопределённое состояние
   LINE_STATE_UP,               // Направление вверх
   LINE_STATE_DOWN,             // Направление вниз
   LINE_STATE_TURN_UP,          // Разворот вверх
   LINE_STATE_TURN_DOWN,        // Разворот вниз
   LINE_STATE_STOP_UP,          // Остановка направления вверх
   LINE_STATE_STOP_DOWN,        // Остановка направления вниз
   LINE_STATE_ABOVE,            // Над значением
   LINE_STATE_BELOW,            // Под значением
   LINE_STATE_CROSS_UP,         // Пересечение значения вверх
   LINE_STATE_CROSS_DOWN,       // Пересечение значения вниз
   LINE_STATE_TOUCH_BELOW,      // Касание значения снизу
   LINE_STATE_TOUCH_ABOVE,      // Касание значения сверху
   LINE_STATE_EQUALS,           // Равно значению
  };
```

Подробнее о перечислении можно прочитать [в статье по индикаторам-осцилляторам в разделе параметров индикатора ATR](#).

Каждый индикатор будет сигнализировать о результате своего расчёта при помощи **кода ошибки**:

```
enum ENUM_ERR_TYPE              // Тип ошибки при расчётах индикаторов
  {
   ERR_TYPE_NO_ERROR,           // Нет ошибки
   ERR_TYPE_NO_CYNC,            // Данные не синхронизированы
   ERR_TYPE_NO_DATA,            // Данные не загружены
   ERR_TYPE_NO_CALC,            // Расчёт не завершён
  };
```

По этому коду можно будет извне определить какое требуется действие для обработки ошибки.

Буферы индикатора

Здесь нам нужно определиться какие буферы к чему относятся.

1. Буфер расчётной части. При создании индикатора, он создаётся в памяти. Это есть расчётная часть индикатора. Она имеет свои буферы, которыми управляет подсистема терминала. Обратиться к расчётной части можно по хэндлу, который возвращается после успешного создания индикатора. В буфере успешно созданного и рассчитанного индикатора всегда есть данные, соответствующие таймсерии, на которой был рассчитан индикатор. Данные в этом буфере располагаются так, что нулевой индекс соответствует текущему бару таймсерии, на которой рассчитан индикатор.
   Для копирования данных из буфера расчётной части индикатора служит функция CopyBuffer().

2. Буфер объекта-индикатора. У каждого из создаваемых объектов классов мульти-индикаторов будут свои массивы-буферы по количеству буферов этого индикатора. В эти массивы будут помещаться данные из буфера расчётной части. Буферы объекта-индикатора будут управляться внутри объекта класса — инициализироваться, увеличивать размер в соответствии с размером таймсерии, на которой создан индикатор, и обновляться на каждом новом тике. При копировании данных в массив объекта-индикатора из буфера расчётной части при помощи CopyBufer(), данные будут располагаться так, что текущий бар будет расположен в конце массива (ArraySize()-1).

3. Буфер рисуемой части индикатора. Каждый объект-индикатор может быть создан как в советнике, так и в пользовательском индикаторе. При создании мульти-индикаторов в советниках, для расчёта индикаторов нужно вызвать метод, рассчитывающий индикатор, а для получения рассчитанных данных — обратиться к нужному индексу буфера объекта-индикатора. А вот в пользовательском индикаторе нужно ещё и нарисовать на графике данные из созданных внутри него мульти-индикаторов. Здесь и появляется ещё один буфер — рисуемый. Это буфер, назначаемый рисуемым в пользовательском индикаторе. В него будут выводиться данные, хранящиеся в буферах объектов-индикаторов. Для вывода линий на график достаточно будет просто из пользовательского индикатора вызвать метод класса-коллекции индикаторов, рассчитывающий индикаторы, а затем, при успешном расчёте, — метод, который расположит данные буфера объекта-индикатора в рисуемом буфере пользовательского индикатора.

В объекте-индикаторе один буфер будет представлен структурой, содержащей как сам динамический массив, так и элементы управления этим массивом:

```
//--- struct
struct SBuffer                  // Структура индикаторного буфера
  {
   double            array[];    // Массив-буфер индикатора
   double            init_value; // Инициализирующее значение
   int               shift;      // Сдвиг буфера по горизонтали
   string            descript;   // Описание буфера
   //--- (1) Устанавливает, (2) возвращает инициализирующее значение,
   void              SetInitValue(const double value) { init_value=value;
   double            InitValue(void)                  { return init_value;
   //--- (1) Устанавливает, (2) возвращает сдвиг буфера
   void              SetShift(const int value)        { shift=value;
   int               Shift(void)                      { return shift;
//--- (1) Изменяет размер массива буфера, (2) возвращает размер массива буфера,
//--- (3) инициализирует массив установленным "пустым" значением
   bool              BuffResize(const int new_size)   { return(ArrayResize(array,new_si
   uint              BufferSize(void)                 { return array.Size();
   int               InitBuffer(void)                 { return ArrayInitialize(array,ir
  };
```

Некоторые значения, установленные извне, например, при создании индикатора, где-то нужно сохранить, чтобы далее знать какими были эти значения. Проще всего записать их прямо в структуру. Именно так здесь и делаем — "пустое" значение буфера, устанавливаемое из вызывающей программы,

сохраняется в структуре буфера в переменной init_value. Сдвиг линии индикатора, который устанавливается при создании расчётной части индикатора, тоже удобно сохранить здесь же, чтобы потом можно было его знать внутри объекта класса, он сохраняется в переменной shift. Здесь же сохраняется и описание буфера, которое задаётся автоматически при создании расчётной части индикатора таким, чтобы соответствовать наименованию буфера у такого же стандартного индикатора. Далее это описание можно будет изменить.

Функции, возвращающие описания состояния линии индикатора и ошибки при расчёте индикатора необходимы лишь для вывода в журнал или на информационную панель состояний линий индикаторов и ошибок при их инициализации и расчёте:

```
//+------------------------------------------------------------------+
//| Возвращает описание состояния линии индикатора                   |
//+------------------------------------------------------------------+
string BufferLineStateDescription(const ENUM_LINE_STATE state)
  {
   switch(state)
     {
      case LINE_STATE_NONE       :  return "None";
      case LINE_STATE_UP         :  return "Up";
      case LINE_STATE_STOP_UP    :  return "Stop Up";
      case LINE_STATE_TURN_UP    :  return "Turn Up";
      case LINE_STATE_DOWN       :  return "Down";
      case LINE_STATE_STOP_DOWN  :  return "Stop Down";
      case LINE_STATE_TURN_DOWN  :  return "Turn Down";
      case LINE_STATE_ABOVE      :  return "Above level";
      case LINE_STATE_BELOW      :  return "Below level";
      case LINE_STATE_CROSS_UP   :  return "Crossing Up";
      case LINE_STATE_CROSS_DOWN :  return "Crossing Down";
      case LINE_STATE_TOUCH_BELOW:  return "Touch from Below";
      case LINE_STATE_TOUCH_ABOVE:  return "Touch from Above";
      case LINE_STATE_EQUALS     :  return "Equals";
      default                    :  return "Unknown";
     }
  }
//+------------------------------------------------------------------+
//| Возвращает описание ошибки при расчёте индикатора                |
//+------------------------------------------------------------------+
string TypeErrorcDescription(ENUM_ERR_TYPE error_type)
  {
   switch(error_type)
     {
      case ERR_TYPE_NO_CYNC   :  return "Data is not synchronized";
      case ERR_TYPE_NO_DATA   :  return "Data not loaded";
      case ERR_TYPE_NO_CALC   :  return "Calculation not completed";
      default                 :  return "No error";
     }
  }
```

Все подготовительные работы сделаны. Займёмся непосредственно классом объекта мультисимвольного мультипериодного индикатора.

Пропишем в теле класса все необходимые для работы класса приватные, защищённые и публичные переменные и методы, а далее рассмотрим их назначение и реализацию:

```
//+------------------------------------------------------------------+
//| Базовый класс мультисимвольного мультипериодного индикатора      |
```

```
//+------------------------------------------------------------------+
class CIndMSTF : public CObject
  {
private:
   ENUM_PROGRAM_TYPE m_program;           // Тип программы
   ENUM_INDICATOR    m_type;              // Тип индикатора
   ENUM_TIMEFRAMES   m_timeframe;         // Период графика
   string            m_symbol;            // Символ графика
   int               m_handle;            // Хэндл индикатора
   int               m_id;                // Идентификатор
   bool              m_success;           // Флаг успешного расчёта
   ENUM_ERR_TYPE     m_type_err;          // Тип ошибки при расчёте
   string            m_description;       // Пользовательское описание индикатора
   string            m_name;              // Наименование индикатора
   string            m_parameters;        // Описание параметров индикатора

protected:
   ENUM_IND_CATEGORY m_category;          // Категория индикатора
   MqlParam          m_param[];           // Массив параметров индикатора
   string            m_title;             // Заголовок (наименование индикатора + описа
   SBuffer           m_buffers[];         // Буферы индикатора
   int               m_digits;            // Digits значений индикатора
   int               m_limit;             // Количество баров, необходимое для просчёта
   int               m_rates_total;       // Количество доступных баров для просчёта ин
   int               m_prev_calculated;   // Количество просчитанных баров на прошлом в

//--- (1) Устанавливает наименование индикатора, (2) описание параметров
   void              SetName(const string name)                    { this.m_name=name
   void              SetParameters(const string str)               { this.m_parameter

//--- Изменяет размер (1) указанного, (2) всех буферов индикатора
   bool              BufferResize(const uint buffer_num,const int new_buff_size);
   bool              BuffersResize(const int new_buff_size);
//--- Инициализирует (1) указанный, (2) все буферы индикатора
   bool              BufferInitialize(const uint buffer_num,const int new_buff_size);
   bool              BuffersInitialize(const int new_buff_size);

//--- Возвращает флаг равенства структуры одного параметра двух объектов
   bool              IsEqualParameters(const MqlParam &this_param,const MqlParam &compa
                       {
                        if(this_param.type==compared_param.type                    &&
                           this_param.integer_value==compared_param.integer_value  &&
                           this_param.string_value==compared_param.string_value    &&
                           ::NormalizeDouble(this_param.double_value-compared_param.dou
                          ) return true;
                        return false;
                       }
//--- Возвращает результат сравнения одного параметра двух объектов
   int               CompareParams(const MqlParam &this_param,const MqlParam &compared_
                       {
                        if(this.IsEqualParameters(this_param,compared_param))
                           return 0;
                        else if(this_param.type>compared_param.type                 ||
                           this_param.integer_value>compared_param.integer_value   ||
                           this_param.string_value>compared_param.string_value     ||
                           this_param.double_value>compared_param.double_value
                          ) return 1;
```

```
                     else if(this_param.type<compared_param.type                 ||
                        this_param.integer_value<compared_param.integer_value    ||
                        this_param.string_value<compared_param.string_value      ||
                        this_param.double_value<compared_param.double_value
                       ) return -1;
                     else
                        return -1;
                    }

public:
//--- Создаёт расчётную часть индикатора, возвращает хэндл
   int               CreateIndicator(void);
//--- (1) Рассчитывает индикатор, (2) заполняет переданный массив-буфер (с учётом симво
   bool              Calculate(void);
   bool              DataToBuffer(const string symbol_to,const ENUM_TIMEFRAMES timefram

//--- (1) Устанавливает (2) возвращает инициализирующее значение для указанного буфера
   void              SetBufferInitValue(const uint buffer_num,const double value);
   double            BufferInitValue(const uint buffer_num) const;

//--- (1) Устанавливает (2) возвращает значение сдвига для указанного буфера
   void              SetBufferShift(const uint buffer_num,const int value);
   double            BufferShift(const uint buffer_num) const;

//--- Возвращает данные указанного буфера (1) как есть, (2) относительно указанного сим
//--- (3) количество данных в указанном буфере, (4) состояние линии индикатора как есть
//--- (5) состояние линии индикатора с учётом символа/периода графика, описание состоян
   double            GetData(const uint buffer_num,const int index)           const;
   double            GetDataTo(const string symbol_to,const ENUM_TIMEFRAMES timeframe_t
   uint              DataTotal(const uint buffer_num)                         const;
   ENUM_LINE_STATE   BufferLineState(const uint buffer_num,const int index)   const;
   ENUM_LINE_STATE   BufferLineState(const string symbol_from,const ENUM_TIMEFRAMES tim
   ENUM_LINE_STATE   BufferLineStateRelative(const int buffer_num,const int index,const
   ENUM_LINE_STATE   BufferLineStateRelative(const string symbol_from,const ENUM_TIMEFR

//--- Возвращает (1) флаг успешности, (2) тип ошибки расчёта
   bool              IsSuccess(void)                          const { return this.m_su
   ENUM_ERR_TYPE     TypeError(void)                          const { return this.m_ty

//--- Устанавливает (1) идентификатор, (2) Digits, (3) пользовательское описание, (4) о
   void              SetID(const int id)                            { this.m_id=id;
   void              SetDigits(const uint digits)                   { this.m_digits=(i
   void              SetDescription(const string descr)             { this.m_descripti
   void              SetBufferDescription(const uint buffer_num,const string descr);

//--- Устанавливает индексацию массивов буферов расчётной части не как в таймсерии
   void              SetAsSeriesOff(void);
//--- Возвращает флаг серийности указанного буфера, (2) синхронизированности историческ
   bool              IsSeries(const uint buffer_num) const;
   bool              IsSynchronized(void) const
                     {
                      return (bool)::SeriesInfoInteger(this.m_symbol,this.m_timeframe
                     }

//--- Возвращает (1) таймфрейм, (2) символ, (3) наименование, (4) список параметров, (5
//--- количество (7) буферов, (8) баров, (9) идентификатор, (10) описание, (11) заголов
//--- (13) количество параметрпов, (14) тип программы, описание (15) категории, (16) бу
```

```
   ENUM_TIMEFRAMES   Timeframe(void)                              const { return this.m_ti
   string            Symbol(void)                               const { return this.m_sy
   string            Name(void)                                 const { return this.m_na
   string            Parameters(void)                           const { return this.m_pa
   int               Handle(void)                               const { return this.m_ha
   int               Digits(void)                               const { return this.m_di
   uint              BuffersTotal(void)                         const { return this.m_bu
   uint              RatesTotal(void)                           const { return this.m_ra
   int               ID(void)                                   const { return this.m_id
   string            Description(void)                          const { return this.m_de
   string            Title(void)                                const { return this.m_ti
   ENUM_IND_CATEGORY Category(void)                             const { return this.m_ca
   uint              ParamsTotal(void)                          const { return this.m_pa
   ENUM_PROGRAM_TYPE Program(void)                              const { return this.m_pr
   string            CategoryDescription(void);
   string            BufferDescription(const uint buffer_num);

//--- Возвращает (1) структуру параметров по индексу из массива, (2) флаг программы-инд
   MqlParam          GetMqlParam(const int index)               const { return this.m_pa
   bool              IsIndicator()                  const { return(this.Program()==PROGR
   string            TimeframeDescription(void)     const
                       {
                        return ::StringSubstr(::EnumToString(this.m_timeframe),7);
                       }
//--- Возвращает количество рассчитанных данных
   int               Calculated(void)  const { return ::BarsCalculated(this.m_handle);

//--- Виртуальный метод, возвращающий тип объекта (индикатора)
   virtual int       Type(void)                                 const { return this.m_ty
//--- Виртуальный метод сравнения двух объектов
   virtual int       Compare(const CObject *node,const int mode=0) const
                       {
                        const CIndMSTF *compared=node;
                        switch(mode)
                          {
                           case COMPARE_MODE_ID          : return(this.ID()>compared.ID
                           case COMPARE_MODE_HANDLE      : return(this.Handle()>compare
                           case COMPARE_MODE_CATEGORY    : return(this.Category()>compa
                           case COMPARE_MODE_SYMBOL      : return(this.Symbol()>compare
                           case COMPARE_MODE_TIMEFRAME   : return(this.Timeframe()>comp
                           case COMPARE_MODE_DESCRIPTION : return(this.Description()>co
                           //---Равенство всех параметров объектов
                           default                       : return(this.IsEqualIndicator
                          }
                       }
//--- Возвращает флаг равенства параметров двух объектов-индикаторов
   bool              IsEqualIndicators(const CIndMSTF *compared) const
                       {
                        if(this.Type()!=compared.Type() || this.ParamsTotal()!=compared
                           return false;
                        bool res=true;
                        int total=(int)this.ParamsTotal();
                        for(int i=0;i<total;i++)
                           res &=this.IsEqualParameters(this.m_param[i],compared.GetMql
                        res &=(this.Timeframe()==compared.Timeframe());
                        res &=(this.Symbol()==compared.Symbol());
                        return res;
```

```
                        }
//--- Таймер
   void OnTimer(void);

//--- Конструктор/деструктор
                       CIndMSTF(){}
                       CIndMSTF(const ENUM_INDICATOR type,const uint buffers,const string
                      ~CIndMSTF();
  };
```

Назначение каждой переменной и методов здесь прокомментированы в коде класса. Рассмотрим реализацию методов.

Конструктор класса:

```
//+------------------------------------------------------------------+
//| Конструктор                                                      |
//+------------------------------------------------------------------+
CIndMSTF::CIndMSTF(const ENUM_INDICATOR type,const uint buffers,const string symbol,cor
  {
//--- Запускаем таймер
   ::ResetLastError();
   if(!::EventSetTimer(1))
      ::PrintFormat("%s: EventSetTimer failed. Error %lu",__FUNCTION__,::GetLastError()
//--- Устанавливаем свойствам переданные в конструктор значения, или значения по умолча
   this.m_program=(ENUM_PROGRAM_TYPE)::MQLInfoInteger(MQL_PROGRAM_TYPE);
   this.m_type=type;
   this.m_symbol=(symbol==NULL || symbol=="" ? ::Symbol() : symbol);
   this.m_timeframe=(timeframe==PERIOD_CURRENT ? ::Period() : timeframe);
   this.m_handle=INVALID_HANDLE;
   this.m_digits=::Digits();
   this.m_success=true;
   this.m_type_err=ERR_TYPE_NO_ERROR;
//--- Устанавливаем массиву структуре буферов размер, равный количеству буферов индикат
   ::ResetLastError();
   if(::ArrayResize(this.m_buffers,buffers)!=buffers)
      ::PrintFormat("%s: Buffers ArrayResize failed. Error  %lu"__FUNCTION__,::GetLastE
//--- Устанавливаем "пустое" значение для каждого буфера по умолчанию (потом можно изме
   for(int i=0;i<(int)this.m_buffers.Size();i++)
      this.SetBufferInitValue(i,EMPTY_VALUE);
//--- Устанавливаем начальные значения переменным, участвующим в экономном расчёте инди
   this.m_prev_calculated=0;
   this.m_limit=0;
   this.m_rates_total=::Bars(this.m_symbol,this.m_timeframe);
//--- Если индикатор рассчитывается не на текущем графике - делаем запрос данных с нужн
//--- (первое обращение к данным запускает подкачку этих данных)
   datetime array[];
   if(symbol!=::Symbol() || timeframe!=::Period())
      ::CopyTime(this.m_symbol,this.m_timeframe,0,this.m_rates_total,array);
  }
```

В конструктор класса передаются тип индикатора, количество его буферов, наименование символа и период графика. Далее переменным устанавливаются значения по умолчанию, устанавливается размер массива буферов, массивам буферов задаётся инициализирующее значение по умолчанию — "пустое" значение как EMPTY_VALUE. Если объект индикатора рассчитывается не на текущем графике, то в конце конструктора делаем обращение к функции, запускающей закачку с сервера данных таймсерии, на каторой рассчитывается индикатор.

Деструктор класса:

```
//+------------------------------------------------------------------+
//| Деструктор                                                       |
//+------------------------------------------------------------------+
CIndMSTF::~CIndMSTF()
  {
//--- Удаляем таймер
   ::EventKillTimer();
//--- Освобождаем хэндл индикатора
   ::ResetLastError();
   if(this.m_handle!=INVALID_HANDLE && !::IndicatorRelease(this.m_handle))
      ::PrintFormat("%s: %s, handle %ld IndicatorRelease failed. Error %ld",__FUNCTION_
//--- Освобождаем память массивов буферов
   for(int i=0;i<(int)this.BuffersTotal();i++)
      ::ArrayFree(this.m_buffers[i].array);
  }
```

В деструкторе класса уничтожается таймер, освобождается хэндл индикатора и память массивов буферов.

Таймер:

```
//+------------------------------------------------------------------+
//| Таймер                                                           |
//+------------------------------------------------------------------+
void CIndMSTF::OnTimer(void)
  {
//--- Если символ и таймфрейм индикатора такие же, как у текущего графика - уходим
   if(this.Symbol()==::Symbol() && this.Timeframe()==::Period())
      return;
//--- Объявляем переменные счётчиков таймера
   static int count1=0;
   static int count2=0;
//--- Если счётчик таймера 1 достиг заданного значения,
   if(count1>=TIMER_COUNT_1)
     {
      //--- вызываем функцию CopyTime (удержание таймсерии) и сбрасываем счётчик
      datetime array[1];
      ::CopyTime(this.m_symbol,this.m_timeframe,0,1,array);
      count1=0;
     }
//--- Если счётчик таймера 2 достиг заданного значения
   if(count2>=TIMER_COUNT_2)
     {
      static int count=0;
      //--- если прошлый расчёт индикатора был неуспешным - эмулируем тик с сообщением
      if(!this.m_success)
        {
         if(::ChartSetSymbolPeriod(0,::Symbol(),::Period()))
           {
            count++;
            ::PrintFormat("%s::%s: Tick emulation. Attempt %ld of 3 ...",__FUNCTION__,t
            if(count>2)
              {
               count=0;
               this.m_success=true;
```

```
               }
            }
         }
      //--- сбрасываем счётчик
      count2=0;
     }
//--- Увеличиваем счётчики таймеров
   count1++;
   count2++;
  }
```

В таймере класса организованы два счётчика: один для удержания таймсерии индикатора, а второй — для эмуляции тиков в выходные дни. Если индикатор рассчитывается на данных текущего символа/периода графика, то таймер не используется.

Сам по себе объект индикатора индикатором как таковым не является. Это лишь обёртка над расчётной частью индикатора, позволяющая управлять ею, получать от неё данные и передавать их в программу. В объекте мульти- индикатора нужно создать сам индикатор. Для этого служит метод для создания расчётной части индикатора, возвращающая полученный при создании хэндл:

```
//+------------------------------------------------------------------+
//| Создаёт индикатор, возвращает хэндл                              |
//+------------------------------------------------------------------+
int CIndMSTF::CreateIndicator(void)
  {
//--- Создаём наименование индикатора для вывода в журнал
   string name=::StringFormat("%s(%s,%s)",::StringSubstr(::EnumToString(this.m_type),4)
//--- Создаём расчётную часть индикатора
   ::ResetLastError();
   this.m_handle=::IndicatorCreate(this.m_symbol,this.m_timeframe,this.m_type,this.m_pa
//--- Если расчётная часть не создана - выводим в журнал сообщение о неудачном создании
   if(this.m_handle==INVALID_HANDLE)
      ::PrintFormat("%s: Failed to create indicator handle %s. Error %ld",__FUNCTION__,
//--- Если индикатор создан - устанавливаем индексацию массивов индикатора не как у тай
   else
      this.SetAsSeriesOff();
//--- Возвращаем хэндл созданного индикатора, либо -1 при неудаче
   return this.m_handle;
  }
```

Буферы объекта-индикатора являются динамическими массивами внутри структуры буфера, рассмотренной нами выше, которым необходимо менять размер под постоянно растущее количество новых данных. Для этого есть два метода: метод для изменения размера массива указанного буфера и для изменения размера сразу всех буферов объекта-индикатора.

Метод, изменяющий размер указанного буфера индикатора:

```
//+------------------------------------------------------------------+
//| Изменяет размер указанного буфера индикатора                     |
//+------------------------------------------------------------------+
bool CIndMSTF::BufferResize(const uint buffer_num,const int new_buff_size)
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
```

```
      return false;
     }
//--- Изменяем размер буфера
   ::ResetLastError();
   bool res=this.m_buffers[buffer_num].BuffResize(new_buff_size);
//--- При неудаче выводим сообщение в журнал
   if(!res)
      ::PrintFormat("%s::%s: Buffer(%lu) resize failed. Error %lu",__FUNCTION__,this.Ti
//--- Возвращаем результат изменения размера массива буфера
   return res;
  }
```

В метод передаётся номер буфера, массив которого нужно изменить на величину, так же передаваемую в метод. При неправильном указании номера буфера об этом выводится запись в журнале и метод возвращает false.
Если размер массива изменить не удалось, то об этом выводится сообщение в журнал. В итоге возвращается результат изменения размера массива буфера.

Метод, изменяющий размер всех буферов индикатора:

```
//+------------------------------------------------------------------+
//| Изменяет размер всех буферов индикатора                          |
//+------------------------------------------------------------------+
bool CIndMSTF::BuffersResize(const int new_buff_size)
  {
//--- В цикле по всем буферам индикатора добавляем к переменной res результат изменения
   bool res=true;
   int total=(int)this.BuffersTotal();
   for(int i=0;i<total;i++)
      res &=this.m_buffers[i].BuffResize(new_buff_size);
//--- Возвращаем результат изменения размеров всех массивов буферов индикатора
   return res;
  }
```

Здесь в цикле по всем буферам объекта-индикатора результат изменения размера массива очередного буфера добавляется к переменной res, значение которой в итоге и возвращается из метода.

Похожим образом организованы методы для инициализации массивов буферов объекта-индикатора.

Метод, инициализирующий указанный буфер индикатора:

```
//+------------------------------------------------------------------+
//| Инициализирует указанный буфер индикатора                        |
//+------------------------------------------------------------------+
bool CIndMSTF::BufferInitialize(const uint buffer_num,const int new_buff_size)
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return false;
     }
//--- Изменяем размер массива буфера
   bool res=this.BufferResize(buffer_num,new_buff_size);
//--- При удачном изменении размера инициализируем буфер установленным инициализирующим
   if(res)
      this.m_buffers[buffer_num].InitBuffer();
```

```
//--- Возвращаем результат
   return res;
  }
```

Здесь тоже проверяется корректность указания номера инициализируемого буфера. Затем буферу устанавливается новый размер и, при успешном его изменении, массив инициализируется установленным для этого буфера значением.

Метод, инициализирующий все буферы индикатора:

```
//+------------------------------------------------------------------+
//| Инициализирует все буферы индикатора                             |
//+------------------------------------------------------------------+
bool CIndMSTF::BuffersInitialize(const int new_buff_size)
  {
//--- В цикле по всем буферам индикатора добавляем к переменной res результат изменения
//--- При удачном изменении размера инициализируем буфер установленным инициализирующим
   bool res=true;
   int total=(int)this.BuffersTotal();
   for(int i=0;i<total;i++)
     {
      res &=this.m_buffers[i].BuffResize(new_buff_size);
      if(res)
         this.m_buffers[i].InitBuffer();
     }
//--- Возвращаем общий результат
   return res;
  }
```

В цикле по всем буферам объекта-индикатора результат изменения размера массива очередного буфера добавляется к переменной res. При успешном изменении размера, буфер инициализируется установленным для него инициализирующим значением. Метод возвращает итоговое состояние переменной res, которая будет иметь значение false в случае, если хотя бы один из буферов не был успешно инициализирован.

Остальные методы установки и возврата значений буферов идентичны вышерассмотренным:

```
//+------------------------------------------------------------------+
//| Устанавливает инициализирующее значение для указанного буфера    |
//+------------------------------------------------------------------+
void CIndMSTF::SetBufferInitValue(const uint buffer_num,const double value)
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return;
     }
//--- Устанавливаем новое инициализирующее значение для указанного буфера
   this.m_buffers[buffer_num].SetInitValue(value);
  }
//+------------------------------------------------------------------+
//| Возвращает инициализирующее значение указанного буфера           |
//+------------------------------------------------------------------+
double CIndMSTF::BufferInitValue(const uint buffer_num) const
  {
```

```
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      //--- Если у индикатора есть буферы, то возвращаем инициализирующее значение само
      return(this.BuffersTotal()>0 ? this.BufferInitValue(0) : EMPTY_VALUE);
     }
//--- Возвращаем инициализирующее значение запрошенного буфера
   return this.m_buffers[buffer_num].InitValue();
  }
//+------------------------------------------------------------------+
//| Устанавливает значение сдвига для указанного буфера              |
//+------------------------------------------------------------------+
void CIndMSTF::SetBufferShift(const uint buffer_num,const int value)
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return;
     }
//--- Устанавливаем значение сдвига для буфера
   this.m_buffers[buffer_num].SetShift(value);
  }
//+------------------------------------------------------------------+
//| Возвращает значение сдвига указанного буфера                     |
//+------------------------------------------------------------------+
double CIndMSTF::BufferShift(const uint buffer_num) const
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      //--- Если у индикатора есть буферы, то возвращаем значение сдвига самого первого
      return(this.BuffersTotal()>0 ? this.m_buffers[0].Shift() : 0);
     }
//--- Возвращаем значение сдвига запрошенного буфера
   return this.m_buffers[buffer_num].Shift();
  }
```

Логика методов подробно расписана в комментариях к коду.

Метод для расчёта данных индикатора. Рассчитанный индикатор (его расчётная часть) имеет буфер, содержащий все рассчитанные данные индикатора. Нам необходимо в массивы-буферы объекта индикатора скопировать данные из буфера расчётной части. При этом необходимо организовать экономный расчёт — копировать целиком весь буфер нужно лишь при первом запуске, либо при изменении в исторических данных.

Такой расчёт мы рассматривали выше, и он должен быть организован и в расчётном методе объекта-индикатора. Тогда при ошибках метод будет возвращать false, а вызывающая программа реагировать на это — выходить из обработчика (OnTick в советнике и OnCalculate в индикаторе) до прихода следующего тика. Ошибками, требующими возврата из метода, будем считать начало загрузки исторических данных, загрузка не всей истории, не полный расчёт индикатора и ошибки копирования данных из буфера

расчётной части в буфер объекта-индикатора. Метод будет записывать код ошибки в переменную — чтобы вызывающая программа могла его прочесть и правильно обработать.

Метод, заполняющий буферы объекта-индикатора данными из буфера расчётной части:

```
//+------------------------------------------------------------------+
//| Заполняет буферы объекта данными из буфера расчётной части       |
//+------------------------------------------------------------------+
bool CIndMSTF::Calculate(void)
  {
//--- Устанавливаем флагу успешности значение true, а типу ошибки - её отсутствие
   this.m_success=true;
   this.m_type_err=ERR_TYPE_NO_ERROR;
//--- Если данные ещё не синхронизированы с торговым сервером,
   if(!this.IsSynchronized())
     {
      //--- Выводим в журнал сообщение о несинхронизированности,
      ::PrintFormat("%s::%s: Waiting for data to sync...",__FUNCTION__,this.Title());
      //--- устанавливаем тип ошибки, к флагу ошибки добавляем false и возвращаем false
      this.m_type_err=ERR_TYPE_NO_CYNC;
      this.m_success &=false;
      return false;
     }
//--- Если метод Calculated вернул -1, это означает начало закачки данных
   if(this.Calculated()==WRONG_VALUE)
     {
      //--- Выводим в журнал сообщение о начале закачки данных,
      ::PrintFormat("%s::%s: Start downloading data by %s/%s. Waiting for the next tick
      //--- устанавливаем тип ошибки, к флагу ошибки добавляем false и возвращаем false
      this.m_type_err=ERR_TYPE_NO_DATA;
      this.m_success &=false;
      return false;
     }
//--- Если метод Calculated вернул 0, это означает, что индикатор ещё не рассчитан
   if(this.Calculated()==0)
     {
      //--- Выводим в журнал сообщение об ожидании расчёта индикатора,
      ::PrintFormat("%s::%s: Waiting for a new tick and when the indicator will be calc
      //--- устанавливаем тип ошибки, к флагу ошибки добавляем false и возвращаем false
      this.m_type_err=ERR_TYPE_NO_CALC;
      this.m_success &=false;
      return false;
     }
//--- Получаем количество баров данных по символу/периоду индикатора
   int bars=::Bars(this.m_symbol,this.m_timeframe);
//--- Если функция Bars вернула нулевое значение, что часто бывает на выходных, рассчит
   if(bars==0)
     {
      //--- Получим дату самого первого доступного бара в истории по символу/периоду
      datetime firstdate=(datetime)::SeriesInfoInteger(this.m_symbol,this.m_timeframe,S
      //--- Получим дату последнего (текущего) бара в истории по символу/периоду
      datetime lastdate=(datetime)::SeriesInfoInteger(this.m_symbol,this.m_timeframe,SE
      //--- Рассчитаем количество баров между первой и последней датами истории
      int sec=::PeriodSeconds(this.m_timeframe);
      ulong date_bars=(((ulong)lastdate-(ulong)firstdate)/(sec>0 ? sec : 1))+1;
      //--- В переменную bars запишем меньшее значение из рассчитанного количества баро
      bars=(int)fmin(date_bars,::TerminalInfoInteger(TERMINAL_MAXBARS));
```

```
      }
//--- Запишем в m_rates_total полученное количество доступных баров
   if(this.m_rates_total!=bars)
      this.m_rates_total=bars;
//--- Если количество доступных баров получено, и их два и меньше,
   if(this.m_rates_total>=0 && this.m_rates_total<3)
     {
      //--- Выведем в журнал сообщение о слишком маленьком количестве доступных баров
      ::PrintFormat("%s::%s: Not enough data for calculation: %ld bars. Waiting for the
      //--- установим тип ошибки, к флагу ошибки добавим false и вернём false
      this.m_type_err=ERR_TYPE_NO_DATA;
      this.m_success &=false;
      return false;
     }

//--- Рассчитаем количество необходимых баров для расчёта индикатора
//--- Либо вся доступная история, либо 1 при открытии нового бара, либо 0 на текущем ти
   this.m_limit=this.m_rates_total-this.m_prev_calculated;
   this.m_prev_calculated=this.Calculated();

//--- Объявляем массив размером 2 для получения в него данных из буфера расчётной части
//--- Получать будем всегда по два бара - прошлый и текущий
   double array[2];
//--- Получаем количество буферов индикатора
   int total=(int)this.BuffersTotal();
//--- Если рассчитанное значение m_limit больше 1 - значит это либо первый запуск, либо
//--- В этом случае необходим полный перерасчёт индикатора
   if(this.m_limit>1)
     {
      //--- В цикле по количеству буферов индикатора
      for(int i=0;i<total;i++)
        {
         //--- изменяем размер массива буфера индикатора и инициализируем его установле
         this.BufferInitialize(i,this.m_rates_total);
         ::ResetLastError();
         //--- Копируем все имеющиеся исторические данные из массива расчётной части ин
         int copied=::CopyBuffer(this.m_handle,i,-this.m_buffers[i].Shift(),this.m_rate
         //--- Если скопированы не все данные
         if(copied!=this.m_rates_total)
           {
            //--- Если CopyBuffer вернула -1, то это означает начало загрузки историчес
            //--- выведем сообщение об этом в журнал
            if(copied==WRONG_VALUE)
               ::PrintFormat("%s::%s: Start downloading data by %s/%s. Waiting for the
            //--- В любом ином случае - ещё не все данные удалось скопировать
            //--- выведем сообщение об этом в журнал
            else
               ::PrintFormat("%s::%s: Not all data was copied. Data available: %lu, tot
            //--- Запишем в тип ошибки отсутствие данных
            this.m_type_err=ERR_TYPE_NO_DATA;
            //--- Добавим к результату значение false и вернём false для выхода из мето
            this.m_success &=false;
            return false;
           }
        }
      //--- Если вышли из цикла копирования всех буферов индикатора, значит всё успешно
      return true;
```

```
      }
//--- Если рассчитанное значение m_limit меньше, либо равно 1 - значит это либо открыти
//--- В этом случае необходимо рассчитать два бара - первый и текущий
   if(this.m_limit<=1)
     {
      //--- В цикле по количеству буферов индикатора
      for(int i=0;i<total;i++)
        {
         //--- Если это открытие нового бара и не удалось изменить размер буфера индика
         if(this.m_limit==1 && !this.BufferResize(i,this.m_rates_total))
           {
            //--- добавим к переменной m_success значение false и вернём false
            //--- При этом сообщение об ошибке будет выведено в журнал из метода Buffer
            this.m_success &=false;
            return false;
           }
         //--- Если не удалось скопировать два бара из буфера расчётной части индикатор
         ::ResetLastError();
         if(::CopyBuffer(this.m_handle,i,-this.m_buffers[i].Shift(),2,array)!=2)
           {
            //--- сообщим об этом в журнал, добавим к переменной m_success значение fal
            ::PrintFormat("%s::%s: CopyBuffer(%lu) failed. Error %lu",__FUNCTION__,this
            this.m_success &=false;
            return false;
           }
         //--- Если дошли сюда, значит копирование успешно -
         //--- скопируем в буфер объекта-индикатора данные из массива array[], в которы
         this.m_buffers[i].array[this.DataTotal(i)-1]=array[1];
         this.m_buffers[i].array[this.DataTotal(i)-2]=array[0];
        }
      //--- Успешно
      return true;
     }
//--- Неопределённый вариант limit - возвращаем false
   return false;
  }
```

Вся логика метода подробно расписана в комментариях к каждому блоку кода. Метод должен вызываться из программы и, если он вернул false, то необходимо выйти из OnTick или OnCalculate до следующего тика.

Если метод отработал без ошибок, то в буфере объекта-индикатора будут содержаться данные, готовые для использования. К ним можно будет обращаться посредством методов, которые будут рассмотрены далее.

В индикаторах есть специальные методы для вывода данных из буфера объекта-индикатора, заполненного этим методом, в рисуемые буферы индикатора. Методы выводят данные в рисуемый буфер либо в том же виде, в котором они записаны в буфере объекта-индикатора, либо с учётом символа/периода графика. Эти методы можно использовать для отображения рассчитанных данных в объекте-индикаторе на текущем графике.

Метод, заполняющий переданный массив данными из буфера класса:

```
//+------------------------------------------------------------------+
//| Заполняет переданный массив данными из буфера класса             |
//+------------------------------------------------------------------+
bool CIndMSTF::DataToBuffer(const string symbol_to,const ENUM_TIMEFRAMES timeframe_to,c
  {
```

```
//--- Устанавливаем флаг успешности
   this.m_success=true;
//--- Получаем направление индексации переданного в метод массива буфера и,
//--- если индексация не как у таймсерии, - устанавливаем индексацию как у таймсерии
   bool as_series=::ArrayGetAsSeries(buffer);
   if(!as_series)
      ::ArraySetAsSeries(buffer,true);
//--- Устанавливаем наименование символа и значение таймфрейма, переданные в метод
   string symbol=(symbol_to=="" || symbol_to==NULL ? ::Symbol() : symbol_to);
   ENUM_TIMEFRAMES timeframe=(timeframe_to==PERIOD_CURRENT ? ::Period() : timeframe_to)
   datetime array[2];
//--- Если это первый запуск, или изменения в истории - инициальзируем массив буфера, п
   if(limit>1 && this.m_limit>1)
     {
      ::PrintFormat("%s::%s First start, or historical data has been changed. Initializ
      ::ArrayInitialize(buffer,this.BufferInitValue(buffer_num));
     }
//--- Устанавливаем значение счётчика цикла (не более максимального количества баров в
   int count=(limit<=1 ? 2 : ::fmin(::TerminalInfoInteger(TERMINAL_MAXBARS),limit));
//--- В цикле от нулевого бара до значения счётчика цикла
   for(int i=0;i<count;i++)
     {
      //--- Если таймфрейм графика совпадает с таймфреймом объекта класса - заполняем б
      if(timeframe==::Period() && this.m_timeframe==::Period())
         buffer[i]=this.GetData(buffer_num,i);
      //--- Иначе, если таймфрейм графика не равен таймфрейму объекта класса
      else
        {
         //--- Узнаём какому времени данного класса принадлежит бар текущего таймфрейма
         ::ResetLastError();
         if(::CopyTime(symbol,timeframe,i,2,array)!=2)
           {
            //--- Если данных нет в терминале - идём дальше
            if(::GetLastError()==4401)
               continue;
            //--- Ошибка получения существующих данных - возвращаем false
            this.m_success &=false;
            return false;
           }
         //--- По времени бара текущего таймфрейма графика находим соответствующий инде
         ::ResetLastError();
         int bar=::iBarShift(this.m_symbol,this.m_timeframe,array[0]);
         if(bar==WRONG_VALUE)
           {
            this.m_success &=false;
            continue;
           }
         //--- Если это исторические данные (не первый и не нулевой бар) -
         //--- записываем в буфер индикатора по индексу цикла значение, полученное из б
         if(i>1)
            buffer[i]=this.GetData(buffer_num,bar);
         //--- Если это текущий (нулевой) или предыдущий (первый) бар
         else
           {
            //--- Получаем время баров 0 и 1 по символу/таймфрейму объекта класса
            if(::CopyTime(this.m_symbol,this.m_timeframe,0,2,array)!=2)
              {
```

```
                this.m_success &=false;
                return false;
               }
             //--- Получаем по времени индексы текущего и предыдущего баров на графике,
             int bar0=::iBarShift(symbol,timeframe,array[1]);
             int bar1=::iBarShift(symbol,timeframe,array[0]);
             if(bar0==WRONG_VALUE || bar1==WRONG_VALUE)
               {
                this.m_success &=false;
                return false;
               }
             //--- Если таймфрейм графика младше таймфрейма объекта класса,
             if(timeframe<this.m_timeframe)
               {
                //--- в цикле от бара с меньшим временем до текущего бара графика заполн
                for(int j=bar1;j>=0;j--)
                   buffer[j]=this.GetData(buffer_num,(j>bar0 ? 1 : 0));
               }
             //--- Если таймфрейм графика старше таймфрейма объекта класса,
             else
               {
                //--- Получаем время текущего и предыдущего баров по символу/таймфрейму
                if(::CopyTime(symbol,timeframe,0,2,array)!=2)
                  {
                   this.m_success &=false;
                   return false;
                  }
                //--- Получаем по времени индексы баров в буфере расчётной части индикат
                int bar0=::iBarShift(this.m_symbol,this.m_timeframe,array[1]);
                int bar1=::iBarShift(this.m_symbol,this.m_timeframe,array[0]);
                //--- Записываем в буфер индикатора по индексам 1 и 0 значения из соотве
                buffer[1]=this.GetData(buffer_num,bar1);
                buffer[0]=this.GetData(buffer_num,bar0);
               }
            }
         }
      }
//--- Устанавливаем изначальную индексацию переданного в метод массива буфера
   ::ArraySetAsSeries(buffer,as_series);
//--- Успешно
   return true;
  }
```

Логика метода подробно расписана в листинге. Суть метода сводится к тому, чтобы верно рассчитать какие бары текущего графика необходимо заполнить данными из массива буфера индикатора, рассчитанного на ином таймфрейме. В метод последним параметром передаётся массив рисуемого буфера пользовательского индикатора, который должен отображать рассчитанный на другом символе/периоде индикатор.

Метод, возвращающий данные указанного буфера как есть:

```
//+------------------------------------------------------------------+
//| Возвращает данные указанного буфера как есть                     |
//+------------------------------------------------------------------+
double CIndMSTF::GetData(const uint buffer_num,const int index) const
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
```

```
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      //--- Если у индикатора есть буферы, то возвращаем "пустое" значение самого перво
      return(this.BuffersTotal()>0 ? this.BufferInitValue(0) : EMPTY_VALUE);
     }
//--- Если указан не правильный индекс - возвращаем "пустое" значение указанного буфера
   if(index<0 || index>(int)this.DataTotal(buffer_num)-1)
      return this.BufferInitValue(buffer_num);
//--- Рассчитываем реальный индекс в массиве-буфере и возвращаем значение по этому инде
   int n=int(this.DataTotal(buffer_num)-1-index);
   return this.m_buffers[buffer_num].array[n];
  }
```

Метод просто возвращает данные из буфера объекта-индикатора по указанному индексу.

Метод, возвращающий данные указанного буфера на указанный символ/таймфрейм:

```
//+------------------------------------------------------------------+
//| Возвращает данные указанного буфера на указанный символ/таймфрейм |
//+------------------------------------------------------------------+
double CIndMSTF::GetDataTo(const string symbol_to,const ENUM_TIMEFRAMES timeframe_to,co
  {
//--- Если указан текущий символ/период графика
   if(timeframe_to==::Period() && this.m_timeframe==::Period() && symbol_to==::Symbol()
      return this.GetData(buffer_num,index);
//--- Узнаём какому времени данного класса принадлежит бар текущего таймфрейма графика,
   datetime array[];
   if(::CopyTime(symbol_to,timeframe_to,index,1,array)!=1)
      return this.BufferInitValue(buffer_num);
//--- По времени бара текущего таймфрейма графика находим соответствующий индекс бара по
   int bar=iBarShift(this.m_symbol,this.m_timeframe,array[0]);
//--- Если бар не найден - возвращаем "пустое" значение, установленное для буфера
   if(bar==WRONG_VALUE)
      return this.BufferInitValue(buffer_num);
//--- Возвращаем значение из буфера объекта-индикатора по найденному индексу
   return this.GetData(buffer_num,bar);
  }
```

Метод находит индекс бара таймсерии, на которой рассчитан индикатор, соответствующий переданным в метод символу/периоду графика и возвращает данные из буфера объекта-индикатора по найденному индексу.

Метод, возвращающий состояние линии индикатора как есть:

```
//+------------------------------------------------------------------+
//| Возвращает состояние линии индикатора как есть                   |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CIndMSTF::BufferLineState(const uint buffer_num,const int index) const
  {
//--- Получаем значения линии индикатора со смещением (0,1,2) относительно переданного
   const double value0=this.GetData(buffer_num,index);
   const double value1=this.GetData(buffer_num,index+1);
   const double value2=this.GetData(buffer_num,index+2);
//--- Если хоть одно из значений получить не удалось - возвращаем неопределённое значени
   if(value0==EMPTY_VALUE || value1==EMPTY_VALUE || value2==EMPTY_VALUE)
      return LINE_STATE_NONE;
```

```
//--- Разворот линии вверх (value2>value1 && value0>value1)
   if(::NormalizeDouble(value2-value1,this.m_digits)>0 && ::NormalizeDouble(value0-valu
      return LINE_STATE_TURN_UP;
//--- Направление линии вверх (value2<=value1 && value0>value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)<=0 && ::NormalizeDouble(value
      return LINE_STATE_UP;
//--- Остановка направления линии вверх (value2<=value1 && value0==value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)<=0 && ::NormalizeDouble(value
      return LINE_STATE_STOP_UP;
//--- Разворот линии вниз (value2<value1 && value0<value1)
   if(::NormalizeDouble(value2-value1,this.m_digits)<0 && ::NormalizeDouble(value0-valu
      return LINE_STATE_TURN_DOWN;
//--- Направление линии вниз (value2>=value1 && value0<value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)>=0 && ::NormalizeDouble(value
      return LINE_STATE_DOWN;
//--- Остановка направления линии вниз (value2>=value1 && value0==value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)>=0 && ::NormalizeDouble(value
      return LINE_STATE_STOP_DOWN;
//--- Неопределённое состояние
   return LINE_STATE_NONE;
  }
```

Метод определяет состояние линии объекта-индикатора по данным в его буфере и возвращает найденное значение.

Метод, возвращающий состояние линии индикатора с учётом символа/периода:

```
//+------------------------------------------------------------------+
//| Возвращает состояние линии индикатора с учётом символа/периода   |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CIndMSTF::BufferLineState(const string symbol_from,const ENUM_TIMEFRAME
  {
//--- Определяем переданные в метод символ/период графика
   string symbol=(symbol_from=="" || symbol_from==NULL ? ::Symbol() : symbol_from);
   ENUM_TIMEFRAMES timeframe=(timeframes_from==PERIOD_CURRENT ? ::Period() : timeframes
//--- Если получаем данные от символа/периода, равные текущему графику - возвращаем сос
   if(symbol==::Symbol() && symbol==this.m_symbol && timeframe==::Period() && timeframe
      return this.BufferLineState(buffer_num,index);
//--- Объявляем переменные для поиска нужных баров на текущем графике
   datetime array[1];
   int      bar0=WRONG_VALUE;
   int      bar1=WRONG_VALUE;
   int      bar2=WRONG_VALUE;

//--- Получаем время первого бара на графике
   ::ResetLastError();
   if(::CopyTime(symbol,timeframe,index,1,array)!=1)
     {
      ::PrintFormat("%s: CopyTime for %s/%s, bar %ld failed. Error %lu",__FUNCTION__,sy
      return LINE_STATE_NONE;
     }
//--- Получаем индекс первого бара в буфере объекта-индикатора по времени открытия бара
   bar0=::iBarShift(this.m_symbol,this.m_timeframe,array[0]);
   if(bar0==WRONG_VALUE)
     {
      ::PrintFormat("%s: iBarShift for %s/%s, time %s failed. Error %lu",__FUNCTION__,t
      return LINE_STATE_NONE;
```

```
      }
//--- Получаем время второго бара на графике
   ::ResetLastError();
   if(::CopyTime(symbol,timeframe,index+1,1,array)!=1)
     {
      ::PrintFormat("%s: CopyTime for %s/%s, bar %ld failed. Error %lu",__FUNCTION__,sy
      return LINE_STATE_NONE;
     }
//--- Получаем индекс второго бара в буфере объекта-индикатора по времени открытия бара
   bar1=::iBarShift(this.m_symbol,this.m_timeframe,array[0]);
   if(bar1==WRONG_VALUE)
     {
      ::PrintFormat("%s: iBarShift for %s/%s, time %s failed. Error %lu",__FUNCTION__,t
      return LINE_STATE_NONE;
     }
//--- Получаем время третьего бара на графике
   ::ResetLastError();
   if(::CopyTime(symbol,timeframe,index+2,1,array)!=1)
     {
      ::PrintFormat("%s: CopyTime for %s/%s, bar %ld failed. Error %lu",__FUNCTION__,sy
      return LINE_STATE_NONE;
     }
//--- Получаем индекс третьего бара в буфере объекта-индикатора по времени открытия бар
   bar2=::iBarShift(this.m_symbol,this.m_timeframe,array[0]);
   if(bar2==WRONG_VALUE)
     {
      ::PrintFormat("%s: iBarShift for %s/%s, time %s failed. Error %lu",__FUNCTION__,t
      return LINE_STATE_NONE;
     }

//--- Получаем значения линии индикатора со смещением (0,1,2) относительно переданного
   const double value0=this.GetData(buffer_num,bar0);
   const double value1=this.GetData(buffer_num,bar1);
   const double value2=this.GetData(buffer_num,bar2);
//--- Если хоть одно из значений получить не удалось - возвращаем неопределённое значен
   if(value0==EMPTY_VALUE || value1==EMPTY_VALUE || value2==EMPTY_VALUE)
      return LINE_STATE_NONE;
//--- Разворот линии вверх (value2>value1 && value0>value1)
   if(::NormalizeDouble(value2-value1,this.m_digits)>0 && ::NormalizeDouble(value0-valu
      return LINE_STATE_TURN_UP;
//--- Направление линии вверх (value2<=value1 && value0>value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)<=0 && ::NormalizeDouble(value
      return LINE_STATE_UP;
//--- Остановка направления линии вверх (value2<=value1 && value0==value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)<=0 && ::NormalizeDouble(value
      return LINE_STATE_STOP_UP;
//--- Разворот линии вниз (value2<value1 && value0<value1)
   if(::NormalizeDouble(value2-value1,this.m_digits)<0 && ::NormalizeDouble(value0-valu
      return LINE_STATE_TURN_DOWN;
//--- Направление линии вниз (value2>=value1 && value0<value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)>=0 && ::NormalizeDouble(value
      return LINE_STATE_DOWN;
//--- Остановка направления линии вниз (value2>=value1 && value0==value1)
   else if(::NormalizeDouble(value2-value1,this.m_digits)>=0 && ::NormalizeDouble(value
      return LINE_STATE_STOP_DOWN;
//--- Неопределённое состояние
```

```
   return LINE_STATE_NONE;
  }
```

Метод позволяет получить состояние линии объекта-индикатора относительно текущего графика. Если объект-индикатиор рассчитан на данных старшего таймфрейма относительно текущего графика, то его линия "растянута" по барам текущего графика. Метод позволяет корректно получить состояние линии индикатора на указанном баре текущего графика.

Метод, возвращающий состояние линии относительно указанного уровня:

```
//+------------------------------------------------------------------+
//| Возвращает состояние линии относительно указанного уровня        |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CIndMSTF::BufferLineStateRelative(const int buffer_num,const int index,
  {
//--- Получаем значения линии индикатора со смещением (0,1) относительно переданного ин
   const double value0=this.GetData(buffer_num,index);
   const double value1=this.GetData(buffer_num,index+1);
//--- Если хоть одно из значений получить не удалось - возвращаем неопределённое значен
   if(value0==EMPTY_VALUE || value1==EMPTY_VALUE)
      return LINE_STATE_NONE;
//--- Определяем второй сравниваемый уровень
   double level=(level1==EMPTY_VALUE ? level0 : level1);
//--- Линия находится под уровнем (value1<level && value0<level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<0 && ::NormalizeDouble(value0-level
      return LINE_STATE_BELOW;
//--- Линия находится над уровнем (value1>level && value0>level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>0 && ::NormalizeDouble(value0-level
      return LINE_STATE_ABOVE;
//--- Линия пересекла уровень снизу-вверх (value1<=level && value0>level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<=0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_CROSS_UP;
//--- Линия пересекла уровень сверху-вниз (value1>=level && value0<level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>=0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_CROSS_DOWN;
//--- Линия коснулась уровня снизу (value1<level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<0 && ::NormalizeDouble(value0-level
      return LINE_STATE_TOUCH_BELOW;
//--- Линия коснулась уровня сверху (value1>level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>0 && ::NormalizeDouble(value0-level
      return LINE_STATE_TOUCH_BELOW;
//--- Линия равна значению уровня (value1==level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)==0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_EQUALS;
//--- Неопределённое состояние
   return LINE_STATE_NONE;
  }
```

Метод, возвращающий состояние линии относительно указанного уровня на указанном символе/периоде графика:

```
//+------------------------------------------------------------------+
//| Возвращает состояние линии относительно указанного уровня        |
//| на указанном символе/периоде графика                             |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CIndMSTF::BufferLineStateRelative(const string symbol_from,const ENUM_T
  {
```

```
//--- Определяем переданные в метод символ/период графика
   string symbol=(symbol_from=="" || symbol_from==NULL ? ::Symbol() : symbol_from);
   ENUM_TIMEFRAMES timeframe=(timeframes_from==PERIOD_CURRENT ? ::Period() : timeframes
//--- Получаем значения линии индикатора со смещением (0,1) относительно переданного ин
   const double value0=this.GetDataTo(symbol,timeframe,buffer_num,index);
   const double value1=this.GetDataTo(symbol,timeframe,buffer_num,index+1);
//--- Если хоть одно из значений получить не удалось - возвращаем неопределённое значен
   if(value0==EMPTY_VALUE || value1==EMPTY_VALUE)
      return LINE_STATE_NONE;
//--- Определяем второй сравниваемый уровень
   double level=(level1==EMPTY_VALUE ? level0 : level1);
//--- Линия находится под уровнем (value1<level && value0<level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<0 && ::NormalizeDouble(value0-level
      return LINE_STATE_BELOW;
//--- Линия находится над уровнем (value1>level && value0>level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>0 && ::NormalizeDouble(value0-level
      return LINE_STATE_ABOVE;
//--- Линия пересекла уровень снизу-вверх (value1<=level && value0>level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<=0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_CROSS_UP;
//--- Линия пересекла уровень сверху-вниз (value1>=level && value0<level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>=0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_CROSS_DOWN;
//--- Линия коснулась уровня снизу (value1<level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)<0 && ::NormalizeDouble(value0-level
      return LINE_STATE_TOUCH_BELOW;
//--- Линия коснулась уровня сверху (value1>level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)>0 && ::NormalizeDouble(value0-level
      return LINE_STATE_TOUCH_BELOW;
//--- Линия равна значению уровня (value1==level0 && value0==level0)
   if(::NormalizeDouble(value1-level,this.m_digits)==0 && ::NormalizeDouble(value0-leve
      return LINE_STATE_EQUALS;
//--- Неопределённое состояние
   return LINE_STATE_NONE;
  }
```

О методах получения состояния линий индикатора можно почитать подробнее в статье о подключении осцилляторов к советникам.

Остальные методы класса:

```
//+------------------------------------------------------------------+
//| Возвращает описание категории                                    |
//+------------------------------------------------------------------+
string CIndMSTF::CategoryDescription(void)
  {
//--- Создаём из перечисления ENUM_IND_CATEGORY наименование категории и возвращаем пол
   string category=::StringSubstr(::EnumToString(this.m_category),13);
   if(category.Lower())
      category.SetChar(0,ushort(category.GetChar(0)-0x20));
   return category;
  }
//+------------------------------------------------------------------+
//| Возвращает описание буфера индикатора                            |
//+------------------------------------------------------------------+
string CIndMSTF::BufferDescription(const uint buffer_num)
  {
```

```
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return "";
     }
//--- Если у индикатора есть буферы, возвращаем описание указанного буфера, иначе - опи
   return(this.BuffersTotal()>0 ? this.m_buffers[buffer_num].descript : this.m_title);
  }
//+------------------------------------------------------------------+
//| Устанавливает описание буфера индикатора                         |
//+------------------------------------------------------------------+
void CIndMSTF::SetBufferDescription(const uint buffer_num,const string descr)
  {
//--- Если индикатор не имеет буферов - уходим
   if(this.BuffersTotal()==0)
      return;
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return;
     }
//--- Записываем в указанный буфер переданное в метод описание
   this.m_buffers[buffer_num].descript=descr;
  }
//+------------------------------------------------------------------+
//| Отключает индексацию массивов буферов как у таймсерии            |
//+------------------------------------------------------------------+
void CIndMSTF::SetAsSeriesOff(void)
  {
//--- В цикле по всем буферам индикатора отключаем флаг серийности массивов
   for(int i=0;i<(int)this.BuffersTotal();i++)
      ::ArraySetAsSeries(this.m_buffers[i].array,false);
  }
//+------------------------------------------------------------------+
//| Возвращает флаг серийности указанного буфера                     |
//+------------------------------------------------------------------+
bool CIndMSTF::IsSeries(const uint buffer_num) const
  {
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return false;
     }
//--- Возвращаем флаг серийности массива указанного буфера
   return (bool)::ArrayGetAsSeries(this.m_buffers[buffer_num].array);
  }
//+------------------------------------------------------------------+
//| Возвращает количество данных указанного буфера                   |
//+------------------------------------------------------------------+
uint CIndMSTF::DataTotal(const uint buffer_num) const
  {
```

```
//--- Проверяем корректность переданного в метод номера буфера и, если номер не верный,
   if(buffer_num>this.BuffersTotal()-1)
     {
      string buff_limit=(this.BuffersTotal()==1 ? "0" : "0 - "+string(this.BuffersTotal
      ::PrintFormat("%s: Invalid buffer number passed (%lu). Value must be %s",__FUNCTI
      return 0;
     }
//--- Возвращаем размер массива указанного буфера
   return this.m_buffers[buffer_num].array.Size();
  }
```

Базовый класс объекта мультисимвольного мультипериодного индикатора готов. Класс содержит весь необходимый функционал для работы с индикаторами, построенными на данных таймсерий, не принадлежащих текущему графику.

Для создания различных типов технических индикаторов нужно создать производные классы от только что созданного базового. В конструкторах производных классов будут указываться те параметры и свойства, которые присущи типу создаваемого индикатора.

Классы индикаторов по типам

Производные от базового классы будут простейшими, и содержать будут только конструктор. В конструктор класса будут передаваться символ/период графика, на которых рассчитывается индикатор, и входные параметры, присущие этому типу индикаторов. В строке инициализации конструктора параметры будут передаваться в конструктор родительского класса.

Пример класса, создающего объект-индикатор, не содержащий никаких параметров:

```
//+------------------------------------------------------------------+
//| Класс индикатора Accelerator Oscillator                          |
//+------------------------------------------------------------------+
class CIndAC : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAC(const string symbol,const ENUM_TIMEFRAMES timeframe) : CIndMSTF(IND_AC,1,symb
     {
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("AC");
      this.SetDescription("Accelerator Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
```

В строке инициализации в родительский класс передаётся тип индикатора, количество буферов, символ и период графика, на которых должен быть рассчитан индикатор. В теле класса создаётся строка с описанием параметров индикатора. В данном случае, если индикатор создаётся на данных текущего

графика, то строка параметров будет пустой. Иначе — в ней будут прописаны символ и период графика в виде, например "(EURUSD,H1)". Далее в теле конструктора устанавливаются все параметры, присущие данному типу индикатора (здесь это индикатор Accelerator Oscillator).

У каждого индикатора есть возможность задать количество знаков после запятой, отображаемых в окне данных и на графике инструмента. В конструкторе этого класса нет установки Digits, так как это значение, равное Digits символа, на котором рассчитывается индикатор, устанавливается в конструкторе родительского класса. Если же необходимо для индикатора установить иное значение Digits, то это либо делается в конструкторах классов тех индикаторов, где Digits значений отличается от Digits символа, либо изменить можно уже после создания объекта-индикатора при помощи метода SetDigits().

Класс индикатора, имеющего параметры:

```
//+------------------------------------------------------------------+
//| Класс индикатора Accumulation/Distribution                       |
//+------------------------------------------------------------------+
class CIndAD : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAD(const string symbol,const ENUM_TIMEFRAMES timeframe,
          const ENUM_APPLIED_VOLUME applied_volume // используемый объем
         ) : CIndMSTF(IND_AD,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("A/D");
      this.SetDescription("Accumulation/Distribution");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_VOLUME;
      this.m_digits=0;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
```

Здесь есть параметры, и их нужно прописать в массив-структуру входных параметров индикатора MqlParam. И здесь же устанавливаем значение Digits индикатора, которое установлено для стандартного Accumulation/Distribution.

Полный список всех классов-наследников базового класса мультисимвольного мультипериодного индикатора:

```
//+------------------------------------------------------------------+
//| Класс индикатора Accelerator Oscillator                          |
//+------------------------------------------------------------------+
class CIndAC : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAC(const string symbol,const ENUM_TIMEFRAMES timeframe) : CIndMSTF(IND_AC,1,symb
     {
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("AC");
      this.SetDescription("Accelerator Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Accumulation/Distribution                       |
//+------------------------------------------------------------------+
class CIndAD : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAD(const string symbol,const ENUM_TIMEFRAMES timeframe,
          const ENUM_APPLIED_VOLUME applied_volume // используемый объем
         ) : CIndMSTF(IND_AD,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("A/D");
      this.SetDescription("Accumulation/Distribution");
```

```
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_VOLUME;
      this.m_digits=0;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Average Directional Movement Index              |
//+------------------------------------------------------------------+
class CIndADX : public CIndMSTF
  {
public:
//--- Конструктор
   CIndADX(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int adx_period                    // период усреднения
          ) : CIndMSTF(IND_ADX,3,symbol,timeframe)
     {
      // Номера буферов: 0 - MAIN_LINE, 1 - PLUSDI_LINE, 2 - MINUSDI_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(adx_period<1 ? 14 : adx_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),adx_peri
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("ADX");
      this.SetDescription("Average Directional Movement Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=2;
      //--- записываем описания буферов линий MAIN_LINE, PLUSDI_LINE и MINUSDI_LINE
      this.SetBufferDescription(MAIN_LINE,this.m_title);
      this.SetBufferDescription(PLUSDI_LINE,"+DI");
      this.SetBufferDescription(MINUSDI_LINE,"-DI");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Average Directional Movement Index Wilder       |
//+------------------------------------------------------------------+
class CIndADXW : public CIndMSTF
  {
public:
//--- Конструктор
   CIndADXW(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int adx_period                    // период усреднения
           ) : CIndMSTF(IND_ADXW,3,symbol,timeframe)
```

```
     {
      // Номера буферов: 0 - MAIN_LINE, 1 - PLUSDI_LINE, 2 - MINUSDI_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(adx_period<1 ? 14 : adx_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),adx_peri
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("ADX Wilder");
      this.SetDescription("Average Directional Movement Index Wilder");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=2;
      //--- записываем описания буферов линий MAIN_LINE, PLUSDI_LINE и MINUSDI_LINE
      this.SetBufferDescription(MAIN_LINE,this.m_title);
      this.SetBufferDescription(PLUSDI_LINE,"+DI");
      this.SetBufferDescription(MINUSDI_LINE,"-DI");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Alligator                                       |
//+------------------------------------------------------------------+
class CIndAlligator : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAlligator(const string symbol,const ENUM_TIMEFRAMES timeframe,
                 const int jaw_period,                  // период для расчета челюстей
                 const int jaw_shift,                   // смещение челюстей по горизо
                 const int teeth_period,                // период для расчета зубов
                 const int teeth_shift,                 // смещение зубов по горизонта
                 const int lips_period,                 // период для расчета губ
                 const int lips_shift,                  // смещение губ по горизонтали
                 const ENUM_MA_METHOD ma_method,        // тип сглаживания
                 const ENUM_APPLIED_PRICE applied_price // тип цены или handle
                ) : CIndMSTF(IND_ALLIGATOR,3,symbol,timeframe)
     {
      // Номера буферов: 0 - GATORJAW_LINE, 1 - GATORTEETH_LINE, 2 - GATORLIPS_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,8)==8)
        {
         ::ZeroMemory(this.m_param);
         //--- период для расчета челюстей
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(jaw_period<1 ? 13 : jaw_period);
```

```
         //--- смещение челюстей по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=jaw_shift;
         //--- период для расчета зубов
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(teeth_period<1 ? 8 : teeth_period);
         //--- смещение зубов по горизонтали
         this.m_param[3].type=TYPE_INT;
         this.m_param[3].integer_value=teeth_shift;
         //--- период для расчета губ
         this.m_param[4].type=TYPE_UINT;
         this.m_param[4].integer_value=(lips_period<1 ? 5 : lips_period);
         //--- смещение губ по горизонтали
         this.m_param[5].type=TYPE_INT;
         this.m_param[5].integer_value=lips_shift;
         //--- тип сглаживания
         this.m_param[6].type=TYPE_UINT;
         this.m_param[6].integer_value=ma_method;
         //--- тип цены или handle
         this.m_param[7].type=TYPE_UINT;
         this.m_param[7].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Alligator");
      this.SetDescription("Alligator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      //--- Записываем описания буферов линий GATORJAW_LINE, GATORTEETH_LINE и GATORLIP
      this.SetBufferDescription(GATORJAW_LINE,::StringFormat("Jaws(%s%lu)", (current ?
      this.SetBufferDescription(GATORTEETH_LINE,::StringFormat("Teeth(%s%lu)",(current
      this.SetBufferDescription(GATORLIPS_LINE,::StringFormat("Lips(%s%lu)", (current ?
      //--- Записываем смещения в буферы GATORJAW_LINE, GATORTEETH_LINE и GATORLIPS_LIN
      this.SetBufferShift(GATORJAW_LINE,jaw_shift);
      this.SetBufferShift(GATORTEETH_LINE,teeth_shift);
      this.SetBufferShift(GATORLIPS_LINE,lips_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Adaptive Moving Average                         |
//+------------------------------------------------------------------+
class CIndAMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int ama_period,                    // период AMA
           const int fast_ma_period,                // период быстрой скользящей
           const int slow_ma_period,                // период медленной скользящей
           const int ama_shift,                     // смещение индикатора по горизонтал
```

```
            const ENUM_APPLIED_PRICE applied_price  // тип цены или handle
           ) : CIndMSTF(IND_AMA,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,5)==5)
        {
         ::ZeroMemory(this.m_param);
         //--- период AMA
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ama_period<1 ? 9 : ama_period);
         //--- период быстрой скользящей
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=(fast_ma_period<1 ? 2 : fast_ma_period);
         //--- период медленной скользящей
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(slow_ma_period<1 ? 30 : slow_ma_period);
         //--- смещение индикатора по горизонтали
         this.m_param[3].type=TYPE_INT;
         this.m_param[3].integer_value=ama_shift;
         //--- тип цены или handle
         this.m_param[4].type=TYPE_UINT;
         this.m_param[4].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("AMA");
      this.SetDescription("Adaptive Moving Average");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ama_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Awesome Oscillator                              |
//+------------------------------------------------------------------+
class CIndAO : public CIndMSTF
  {
public:
//--- Конструктор
   CIndAO(const string symbol,const ENUM_TIMEFRAMES timeframe) : CIndMSTF(IND_AO,1,symb
     {
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
```

```
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("AO");
      this.SetDescription("Awesome Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Average True Range                              |
//+------------------------------------------------------------------+
class CIndATR : public CIndMSTF
  {
public:
//--- Конструктор
   CIndATR(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int ma_period                      // период усреднения
          ) : CIndMSTF(IND_ATR,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("ATR");
      this.SetDescription("Average True Range");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Bears Power                                     |
//+------------------------------------------------------------------+
class CIndBears : public CIndMSTF
  {
public:
//--- Конструктор
   CIndBears(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int ma_period                        // период усреднения
```

```
            ) : CIndMSTF(IND_BEARS,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 13 : ma_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Bears");
      this.SetDescription("Bears Power");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Bulls Power                                     |
//+------------------------------------------------------------------+
class CIndBulls : public CIndMSTF
  {
public:
//--- Конструктор
   CIndBulls(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int ma_period                       // период усреднения
            ) : CIndMSTF(IND_BULLS,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 13 : ma_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Bulls");
```

```
      this.SetDescription("Bulls Power");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Bollinger Bands®                                |
//+------------------------------------------------------------------+
class CIndBands : public CIndMSTF
  {
public:
//--- Конструктор
   CIndBands(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int bands_period,                  // период для расчета средней лин
             const int bands_shift,                   // смещение индикатора по горизон
             const double deviation,                  // кол-во стандартных отклонений
             const ENUM_APPLIED_PRICE applied_price   // тип цены или handle
            ) : CIndMSTF(IND_BANDS,3,symbol,timeframe)
     {
      // Номера буферов: 0 - BASE_LINE, 1 - UPPER_BAND, 2 - LOWER_BAND
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,4)==4)
        {
         ::ZeroMemory(this.m_param);
         //--- период для расчета средней линии
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(bands_period<1 ? 20 : bands_period);
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=bands_shift;
         //--- кол-во стандартных отклонений
         this.m_param[2].type=TYPE_DOUBLE;
         this.m_param[2].double_value=deviation;
         //--- тип цены или handle
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),bands_pe
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Bands");
      this.SetDescription("Bollinger Bands");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //---Описание буферов линий BASE_LINE, UPPER_BAND и LOWER_BAND
      this.SetBufferDescription(BASE_LINE,this.m_title+" Middle");
```

```
      this.SetBufferDescription(UPPER_BAND,this.m_title+" Upper");
      this.SetBufferDescription(LOWER_BAND,this.m_title+" Lower");
      //--- Записываем смещения в буферы BASE_LINE, UPPER_BAND и LOWER_BAND
      this.SetBufferShift(BASE_LINE,bands_shift);
      this.SetBufferShift(UPPER_BAND,bands_shift);
      this.SetBufferShift(LOWER_BAND,bands_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Commodity Channel Index                         |
//+------------------------------------------------------------------+
class CIndCCI : public CIndMSTF
  {
public:
//--- Конструктор
   CIndCCI(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int ma_period,                    // период усреднения
           const ENUM_APPLIED_PRICE applied_price  // тип цены или handle
          ) : CIndMSTF(IND_CCI,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period<2 ? 2 : ma_period)
         //--- тип цены или handle
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("CCI");
      this.SetDescription("Commodity Channel Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=2;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Chaikin Oscillator                              |
//+------------------------------------------------------------------+
class CIndCHO : public CIndMSTF
  {
public:
//--- Конструктор
```

```
   CIndCHO(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int fast_ma_period,                  // быстрый период
           const int slow_ma_period,                  // медленный период
           const ENUM_MA_METHOD ma_method,            // тип сглаживания
           const ENUM_APPLIED_VOLUME applied_volume   // используемый объем
          ) : CIndMSTF(IND_CHAIKIN,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,4)==4)
        {
         ::ZeroMemory(this.m_param);
         //--- быстрый период
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(fast_ma_period<1 ? 3 : fast_ma_period);
         //--- медленный период
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=(slow_ma_period<1 ? 10 : slow_ma_period);
         //--- тип сглаживания
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=ma_method;
         //--- используемый объем
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu)",symbol_period,(current ? "" : ":"),slov
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("CHO");
      this.SetDescription("Chaikin Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=0;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Double Exponential Moving Average               |
//+------------------------------------------------------------------+
class CIndDEMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndDEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int ma_period,                      // период усреднения
            const int ma_shift,                       // смещение индикатора по горизон
            const ENUM_APPLIED_PRICE applied_price    // тип цены или handle
           ) : CIndMSTF(IND_DEMA,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
```

```
      ::ResetLastError();
      if(::ArrayResize(this.m_param,3)==3)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=ma_shift;
         //--- тип цены или handle
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("DEMA");
      this.SetDescription("Double Exponential Moving Average");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ma_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора DeMarker                                        |
//+------------------------------------------------------------------+
class CIndDeM : public CIndMSTF
  {
public:
//--- Конструктор
   CIndDeM(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int ma_period                      // период усреднения
          ) : CIndMSTF(IND_DEMARKER,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
```

```
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("DeM");
      this.SetDescription("DeMarker");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=3;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Envelopes                                       |
//+------------------------------------------------------------------+
class CIndEnvelopes : public CIndMSTF
  {
public:
//--- Конструктор
   CIndEnvelopes(const string symbol,const ENUM_TIMEFRAMES timeframe,
                 const int ma_period,                    // период для расчета средней
                 const int ma_shift,                     // смещение индикатора по гори
                 const ENUM_MA_METHOD ma_method,         // тип сглаживания
                 const ENUM_APPLIED_PRICE applied_price, // тип цены или handle
                 const double deviation                  // отклонение границ от средне
          ) : CIndMSTF(IND_ENVELOPES,2,symbol,timeframe)
     {
      // Номера буферов: 0 - UPPER_LINE, 1 - LOWER_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,5)==5)
        {
         ::ZeroMemory(this.m_param);
         //--- период для расчета средней линии
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=ma_shift;
         //--- тип сглаживания
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=ma_method;
         //--- тип цены или handle
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_price;
         //--- отклонение границ от средней линии
         this.m_param[4].type=TYPE_UINT;
         this.m_param[4].double_value=deviation;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
```

```
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Envelopes");
      this.SetDescription("Envelopes");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Описание буферов линий UPPER_LINE и LOWER_LINE
      this.SetBufferDescription(UPPER_LINE,this.m_title+" Upper");
      this.SetBufferDescription(LOWER_LINE,this.m_title+" Lower");
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ma_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Force Index                                     |
//+------------------------------------------------------------------+
class CIndForce : public CIndMSTF
  {
public:
//--- Конструктор
   CIndForce(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int                 ma_period,     // период усреднения
             const ENUM_MA_METHOD      ma_method,     // тип сглаживания
             const ENUM_APPLIED_VOLUME applied_volume // тип объема для расчета
            ) : CIndMSTF(IND_FORCE,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,3)==3)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 13 : ma_period);
         //--- тип сглаживания
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=ma_method;
         //--- тип объема для расчета
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Force");
      this.SetDescription("Force Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
```

```
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Fractals                                        |
//+------------------------------------------------------------------+
class CIndFractals : public CIndMSTF
  {
public:
//--- Конструктор
   CIndFractals(const string symbol,const ENUM_TIMEFRAMES timeframe) : CIndMSTF(IND_FRA
     {
      // Номера буферов: 0 - UPPER_LINE, 1 - LOWER_LINE
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Fractals");
      this.SetDescription("Fractals");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      //--- Описание буферов линий UPPER_LINE и LOWER_LINE
      this.SetBufferDescription(UPPER_LINE,this.m_title+" Up");
      this.SetBufferDescription(LOWER_LINE,this.m_title+" Down");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Fractal Adaptive Moving Average                 |
//+------------------------------------------------------------------+
class CIndFrAMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndFrAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int ma_period,                    // период усреднения
             const int ma_shift,                     // смещение индикатора по горизон
             const ENUM_APPLIED_PRICE applied_price  // тип цены или handle
            ) : CIndMSTF(IND_FRAMA,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,3)==3)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=ma_shift;
         //--- тип цены или handle
         this.m_param[2].type=TYPE_UINT;
```

```
         this.m_param[2].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("FRAMA");
      this.SetDescription("Fractal Adaptive Moving Average");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ma_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Gator Oscillator                                |
//+------------------------------------------------------------------+
class CIndGator : public CIndMSTF
  {
public:
//--- Конструктор
   CIndGator(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int jaw_period,                    // период для расчета челюстей
             const int jaw_shift,                     // смещение челюстей по горизонта
             const int teeth_period,                  // период для расчета зубов
             const int teeth_shift,                   // смещение зубов по горизонтали
             const int lips_period,                   // период для расчета губ
             const int lips_shift,                    // смещение губ по горизонтали
             const ENUM_MA_METHOD ma_method,          // тип сглаживания
             const ENUM_APPLIED_PRICE applied_price   // тип цены или handle
            ) : CIndMSTF(IND_GATOR,4,symbol,timeframe)
     {
      // Номера буферов: 0 - UPPER_HISTOGRAM, 1- цветовой буфер верхней гистограммы, 2
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,8)==8)
        {
         ::ZeroMemory(this.m_param);
         //--- период для расчета челюстей
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(jaw_period<1 ? 13 : jaw_period);
         //--- смещение челюстей по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=jaw_shift;
         //--- период для расчета зубов
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(teeth_period<1 ? 8 : teeth_period);
         //--- смещение зубов по горизонтали
         this.m_param[3].type=TYPE_INT;
```

```
         this.m_param[3].integer_value=teeth_shift;
         //--- период для расчета губ
         this.m_param[4].type=TYPE_UINT;
         this.m_param[4].integer_value=(lips_period<1 ? 5 : lips_period);
         //--- смещение губ по горизонтали
         this.m_param[5].type=TYPE_INT;
         this.m_param[5].integer_value=lips_shift;
         //--- тип сглаживания
         this.m_param[6].type=TYPE_UINT;
         this.m_param[6].integer_value=ma_method;
         //--- тип цены или handle
         this.m_param[7].type=TYPE_UINT;
         this.m_param[7].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Gator");
      this.SetDescription("Gator Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      this.m_digits=::Digits()+1;
      //--- Описание буферов линий UPPER_HISTOGRAM, буфер цвета верхней гистограммы, LO
      this.SetBufferDescription(UPPER_HISTOGRAM,this.m_title+" Up");
      this.SetBufferDescription(1,this.m_title+" Colors Up");
      this.SetBufferDescription(LOWER_HISTOGRAM,this.m_title+" Down");
      this.SetBufferDescription(3,this.m_title+" Colors Down");
      //--- Записываем смещения в буферы UPPER_HISTOGRAM, 1, LOWER_HISTOGRAM и 2
      this.SetBufferShift(UPPER_HISTOGRAM,teeth_shift);
      this.SetBufferShift(1,teeth_shift);
      this.SetBufferShift(LOWER_HISTOGRAM,lips_shift);
      this.SetBufferShift(3,lips_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Ichimoku Kinko Hyo                              |
//+------------------------------------------------------------------+
class CIndIchimoku : public CIndMSTF
  {
public:
//--- Конструктор
   CIndIchimoku(const string symbol,const ENUM_TIMEFRAMES timeframe,
                const int tenkan_sen,                    // период Tenkan-sen
                const int kijun_sen,                     // период Kijun-sen
                const int senkou_span_b                  // период Senkou Span B
               ) : CIndMSTF(IND_ICHIMOKU,5,symbol,timeframe)
     {
      // Номера буферов: 0 - TENKANSEN_LINE, 1 - KIJUNSEN_LINE, 2 - SENKOUSPANA_LINE, 3
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,3)==3)
```

```
        {
         ::ZeroMemory(this.m_param);
         //--- период Tenkan-sen
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(tenkan_sen<1 ? 9 : tenkan_sen);
         //--- период Kijun-sen
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=(kijun_sen<1 ? 26 : kijun_sen);
         //--- период Senkou Span B
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(senkou_span_b<1 ? 52 : senkou_span_b);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Ichimoku");
      this.SetDescription("Ichimoku Kinko Hyo");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Описание буферов линий TENKANSEN_LINE, KIJUNSEN_LINE, SENKOUSPANA_LINE, SEN
      this.SetBufferDescription(TENKANSEN_LINE,::StringFormat("Tenkan-sen(%lu)",tenkan_
      this.SetBufferDescription(KIJUNSEN_LINE,::StringFormat("Kijun-sen(%lu)",kijun_ser
      this.SetBufferDescription(SENKOUSPANA_LINE,"Senkou Span A");
      this.SetBufferDescription(SENKOUSPANB_LINE,::StringFormat("Senkou Span B(%lu)",se
      this.SetBufferDescription(CHIKOUSPAN_LINE,"Chikou Span");
      //--- Записываем смещения в буферы SENKOUSPANA_LINE, SENKOUSPANB_LINE и CHIKOUSPA
      //this.SetBufferShift(SENKOUSPANA_LINE,kijun_sen);
      //this.SetBufferShift(SENKOUSPANB_LINE,kijun_sen);
      //this.SetBufferShift(CHIKOUSPAN_LINE,kijun_sen-senkou_span_b);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Market Facilitation Index                       |
//+------------------------------------------------------------------+
class CIndBWMFI : public CIndMSTF
  {
public:
//--- Конструктор
   CIndBWMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const ENUM_APPLIED_VOLUME applied_volume // тип объема для расчета
            ) : CIndMSTF(IND_BWMFI,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         //--- тип объема для расчета
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=applied_volume;
```

```
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("BW MFI");
      this.SetDescription("Market Facilitation Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_WILLIAMS;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Momentum                                        |
//+------------------------------------------------------------------+
class CIndMomentum : public CIndMSTF
  {
public:
//--- Конструктор
   CIndMomentum(const string symbol,const ENUM_TIMEFRAMES timeframe,
                const int                 mom_period,    // период усреднения
                const ENUM_APPLIED_PRICE  applied_price  // тип цены или handle
               ) : CIndMSTF(IND_MOMENTUM,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(mom_period<1 ? 14 : mom_period);
         //--- тип цены или handle
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),mom_peri
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("Momentum");
      this.SetDescription("Momentum");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=2;
      //--- Записываем описание буфера линии
```

```
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Money Flow Index                                |
//+------------------------------------------------------------------+
class CIndMFI : public CIndMSTF
  {
public:
//--- Конструктор
   CIndMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int                 ma_period,       // период усреднения
           const ENUM_APPLIED_VOLUME applied_volume   // тип объема для расчета
          ) : CIndMSTF(IND_MFI,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
         //--- тип объема для расчета
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("MFI");
      this.SetDescription("Money Flow Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_VOLUME;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Moving Average                                  |
//+------------------------------------------------------------------+
class CIndMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
          const int                 ma_period,     // период усреднения
          const int                 ma_shift,      // смещение индикатора по горизонтал
          const ENUM_MA_METHOD      ma_method,     // тип сглаживания
          const ENUM_APPLIED_PRICE  applied_price  // тип цены или handle
         ) : CIndMSTF(IND_MA,1,symbol,timeframe)
```

```
    {
     //--- Устанавливаем размер массива параметров и заполняем его
     ::ResetLastError();
     if(::ArrayResize(this.m_param,4)==4)
       {
        ::ZeroMemory(this.m_param);
        //--- период усреднения
        this.m_param[0].type=TYPE_UINT;
        this.m_param[0].integer_value=(ma_period<1 ? 10 : ma_period);
        //--- смещение индикатора по горизонтали
        this.m_param[1].type=TYPE_INT;
        this.m_param[1].integer_value=ma_shift;
        //--- тип сглаживания
        this.m_param[2].type=TYPE_UINT;
        this.m_param[2].integer_value=ma_method;
        //--- тип цены или handle
        this.m_param[3].type=TYPE_UINT;
        this.m_param[3].integer_value=applied_price;
       }
     else
        ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
     //--- Создаём описание параметров.
     //--- Если символ или период графика не текущие, то к параметрам добавляются их о
     bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
     string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
     string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
     //--- Записываем описание параметров, наименование индикатора, его описание, заго
     this.SetParameters(param);
     this.SetName("MA");
     this.SetDescription("Moving Average");
     this.m_title=this.Name()+this.Parameters();
     this.m_category=IND_CATEGORY_TREND;
     this.m_digits=::Digits()+1;
     //--- Записываем описание буфера линии
     this.SetBufferDescription(0,this.m_title);
     //--- Записываем смещение в буфер 0
     this.SetBufferShift(0,ma_shift);
    }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Moving Average of Oscillator                    |
//+------------------------------------------------------------------+
class CIndOsMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndOsMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int                fast_ema_period, // период быстрой средней
            const int                slow_ema_period, // период медленной средней
            const int                signal_period,   // период усреднения разности
            const ENUM_APPLIED_PRICE applied_price    // тип цены или handle
           ) : CIndMSTF(IND_OSMA,1,symbol,timeframe)
    {
     //--- Устанавливаем размер массива параметров и заполняем его
     ::ResetLastError();
     if(::ArrayResize(this.m_param,4)==4)
       {
```

```
         ::ZeroMemory(this.m_param);
         //--- период быстрой средней
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(fast_ema_period<1 ? 12 : fast_ema_period);
         //--- период медленной средней
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=(slow_ema_period<1 ? 26 : slow_ema_period);
         //--- период усреднения разности
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(signal_period<1 ? 9 : signal_period<2 ? 2 : sig
         //--- тип цены или handle
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("OsMA");
      this.SetDescription("Moving Average of Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=::Digits()+2;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Moving Averages Convergence/Divergence          |
//+------------------------------------------------------------------+
class CIndMACD : public CIndMSTF
  {
public:
//--- Конструктор
   CIndMACD(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int                fast_ema_period, // период быстрой средней
            const int                slow_ema_period, // период медленной средней
            const int                signal_period,   // период усреднения разности
            const ENUM_APPLIED_PRICE applied_price    // тип цены или handle
           ) : CIndMSTF(IND_MACD,2,symbol,timeframe)
     {
      // Номера буферов: 0 - MAIN_LINE, 1 - SIGNAL_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,4)==4)
        {
         ::ZeroMemory(this.m_param);
         //--- период быстрой средней
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(fast_ema_period<1 ? 12 : fast_ema_period);
         //--- период медленной средней
         this.m_param[1].type=TYPE_UINT;
```

```
         this.m_param[1].integer_value=(slow_ema_period<1 ? 26 : slow_ema_period);
         //--- период усреднения разности
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(signal_period<1 ? 9 : signal_period);
         //--- тип цены или handle
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("MACD");
      this.SetDescription("Moving Averages Convergence/Divergence");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=::Digits()+1;
      //--- Описание буферов линии MAIN_LINE и SIGNAL_LINE
      this.SetBufferDescription(MAIN_LINE,this.m_title);
      this.SetBufferDescription(SIGNAL_LINE,"Signal");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора On Balance Volume                               |
//+------------------------------------------------------------------+
class CIndOBV : public CIndMSTF
  {
public:
//--- Конструктор
   CIndOBV(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const ENUM_APPLIED_VOLUME applied_volume   // тип объема для расчета
          ) : CIndMSTF(IND_OBV,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         //--- тип объема для расчета
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=(current ? "" : StringFormat("(%s)",symbol_period));
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("OBV");
```

```
      this.SetDescription("On Balance Volume");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_VOLUME;
      this.m_digits=0;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Parabolic Stop and Reverse system               |
//+------------------------------------------------------------------+
class CIndSAR : public CIndMSTF
  {
public:
//--- Конструктор
   CIndSAR(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const double step,                     // шаг изменения цены - коэффициент
           const double maximum                   // максимальный шаг
          ) : CIndMSTF(IND_SAR,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- шаг изменения цены - коэффициент ускорения
         this.m_param[0].type=TYPE_DOUBLE;
         this.m_param[0].double_value=step;
         //--- максимальный шаг
         this.m_param[1].type=TYPE_DOUBLE;
         this.m_param[1].double_value=maximum;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%.2f,%.2f)",symbol_period,(current ? "" : ":"),st
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("SAR");
      this.SetDescription("Parabolic SAR");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Relative Strength Index                         |
//+------------------------------------------------------------------+
class CIndRSI : public CIndMSTF
  {
public:
//--- Конструктор
   CIndRSI(const string symbol,const ENUM_TIMEFRAMES timeframe,
```

```
            const int                ma_period,     // период усреднения
            const ENUM_APPLIED_PRICE applied_price  // тип цены или handle
           ) : CIndMSTF(IND_RSI,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period<2 ? 2 : ma_period)
         //--- тип цены или handle
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("RSI");
      this.SetDescription("Relative Strength Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=2;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Relative Vigor Index                            |
//+------------------------------------------------------------------+
class CIndRVI : public CIndMSTF
  {
public:
//--- Конструктор
   CIndRVI(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int ma_period                    // период усреднения
           ) : CIndMSTF(IND_RVI,2,symbol,timeframe)
     {
      // Номера буферов: 0 - MAIN_LINE, 1 - SIGNAL_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 10 : ma_period);
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
```

```
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("RVI");
      this.SetDescription("Relative Vigor Index");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=3;
      //--- Описание буферов линии MAIN_LINE и SIGNAL_LINE
      this.SetBufferDescription(MAIN_LINE,this.m_title);
      this.SetBufferDescription(SIGNAL_LINE,"Signal");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Standard Deviation                              |
//+------------------------------------------------------------------+
class CIndStdDev : public CIndMSTF
  {
public:
//--- Конструктор
   CIndStdDev(const string symbol,const ENUM_TIMEFRAMES timeframe,
              const int                ma_period,        // период усреднения
              const int                ma_shift,         // смещение индикатора по гори
              const ENUM_MA_METHOD     ma_method,        // тип сглаживания
              const ENUM_APPLIED_PRICE applied_price     // тип цены или handle
             ) : CIndMSTF(IND_STDDEV,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,4)==4)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 20 : ma_period<2 ? 2 : ma_period)
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=ma_shift;
         //--- тип сглаживания
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=ma_method;
         //--- тип цены или handle
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
```

```
      this.SetParameters(param);
      this.SetName("StdDev");
      this.SetDescription("Standard Deviation");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ma_shift);
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Stochastic Oscillator                           |
//+------------------------------------------------------------------+
class CIndStoch : public CIndMSTF
  {
public:
//--- Конструктор
   CIndStoch(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int              Kperiod,          // K-период (количество баров для
             const int              Dperiod,          // D-период (период первичного сг
             const int              slowing,          // окончательное сглаживание
             const ENUM_MA_METHOD   ma_method,        // тип сглаживания
             const ENUM_STO_PRICE   price_field       // способ расчета стохастика
            ) : CIndMSTF(IND_STOCHASTIC,2,symbol,timeframe)
     {
      // Номера буферов: 0 - MAIN_LINE, 1 - SIGNAL_LINE
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,5)==5)
        {
         ::ZeroMemory(this.m_param);
         //--- K-период (количество баров для расчетов)
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(Kperiod<1 ? 5 : Kperiod);
         //--- D-период (период первичного сглаживания)
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=(Dperiod<1 ? 3 : Dperiod);
         //--- окончательное сглаживание
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=(slowing<1 ? 3 : slowing);
         //--- тип сглаживания
         this.m_param[3].type=TYPE_UINT;
         this.m_param[3].integer_value=ma_method;
         //--- способ расчета стохастика
         this.m_param[4].type=TYPE_UINT;
         this.m_param[4].integer_value=price_field;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu,%lu,%lu)",symbol_period,(current ? "" : ":"),
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
```

```
      this.SetParameters(param);
      this.SetName("Stoch");
      this.SetDescription("Stochastic Oscillator");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      this.m_digits=2;
      //--- Описание буферов линии MAIN_LINE и SIGNAL_LINE
      this.SetBufferDescription(MAIN_LINE,this.m_title);
      this.SetBufferDescription(SIGNAL_LINE,"Signal");
     }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Triple Exponential Moving Average               |
//+------------------------------------------------------------------+
class CIndTEMA : public CIndMSTF
  {
public:
//--- Конструктор
   CIndTEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int                  ma_period,     // период усреднения
            const int                  ma_shift,      // смещение индикатора по горизон
            const ENUM_APPLIED_PRICE   applied_price  // тип цены или handle
           ) : CIndMSTF(IND_TEMA,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,3)==3)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period);
         //--- смещение индикатора по горизонтали
         this.m_param[1].type=TYPE_INT;
         this.m_param[1].integer_value=ma_shift;
         //--- тип цены или handle
         this.m_param[2].type=TYPE_UINT;
         this.m_param[2].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("TEMA");
      this.SetDescription("Triple Exponential Moving Average");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_TREND;
      this.m_digits=::Digits()+1;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
      //--- Записываем смещение в буфер 0
      this.SetBufferShift(0,ma_shift);
```

```
      }
   };
//+------------------------------------------------------------------+
//| Класс индикатора Triple Exponential Moving Averages Oscillator   |
//+------------------------------------------------------------------+
class CIndTriX : public CIndMSTF
   {
public:
//--- Конструктор
   CIndTriX(const string symbol,const ENUM_TIMEFRAMES timeframe,
            const int                ma_period,       // период усреднения
            const ENUM_APPLIED_PRICE applied_price    // тип цены или handle
           ) : CIndMSTF(IND_TRIX,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,2)==2)
        {
         ::ZeroMemory(this.m_param);
         //--- период усреднения
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=(ma_period<1 ? 14 : ma_period<2 ? 2 : ma_period)
         //--- тип цены или handle
         this.m_param[1].type=TYPE_UINT;
         this.m_param[1].integer_value=applied_price;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
      //--- Создаём описание параметров.
      //--- Если символ или период графика не текущие, то к параметрам добавляются их о
      bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
      string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
      string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),ma_perio
      //--- Записываем описание параметров, наименование индикатора, его описание, заго
      this.SetParameters(param);
      this.SetName("TRIX");
      this.SetDescription("Triple Exponential Average");
      this.m_title=this.Name()+this.Parameters();
      this.m_category=IND_CATEGORY_OSCILLATOR;
      //--- Записываем описание буфера линии
      this.SetBufferDescription(0,this.m_title);
     }
   };
//+------------------------------------------------------------------+
//| Класс индикатора Larry Williams' Percent Range                   |
//+------------------------------------------------------------------+
class CIndWPR : public CIndMSTF
   {
public:
//--- Конструктор
   CIndWPR(const string symbol,const ENUM_TIMEFRAMES timeframe,
           const int calc_period                    // период усреднения
          ) : CIndMSTF(IND_WPR,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
```

```
      {
       ::ZeroMemory(this.m_param);
       //--- период усреднения
       this.m_param[0].type=TYPE_UINT;
       this.m_param[0].integer_value=(calc_period<1 ? 14 : calc_period);
      }
    else
       ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
    //--- Создаём описание параметров.
    //--- Если символ или период графика не текущие, то к параметрам добавляются их о
    bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
    string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
    string param=StringFormat("(%s%s%lu)",symbol_period,(current ? "" : ":"),calc_per
    //--- Записываем описание параметров, наименование индикатора, его описание, заго
    this.SetParameters(param);
    this.SetName("%R");
    this.SetDescription("Williams' Percent Range");
    this.m_title=this.Name()+this.Parameters();
    this.m_category=IND_CATEGORY_OSCILLATOR;
    this.m_digits=2;
    //--- Записываем описание буфера линии
    this.SetBufferDescription(0,this.m_title);
   }
 };
//+------------------------------------------------------------------+
//| Класс индикатора Variable Index Dynamic Average                  |
//+------------------------------------------------------------------+
class CIndVIDyA : public CIndMSTF
 {
public:
//--- Конструктор
   CIndVIDyA(const string symbol,const ENUM_TIMEFRAMES timeframe,
             const int                cmo_period,    // период Chande Momentum
             const int                ema_period,    // период фактора сглаживания
             const int                ma_shift,      // смещение индикатора по горизон
             const ENUM_APPLIED_PRICE  applied_price  // тип цены или handle
            ) : CIndMSTF(IND_VIDYA,1,symbol,timeframe)
    {
     //--- Устанавливаем размер массива параметров и заполняем его
     ::ResetLastError();
     if(::ArrayResize(this.m_param,4)==4)
       {
        ::ZeroMemory(this.m_param);
        //--- период Chande Momentum
        this.m_param[0].type=TYPE_UINT;
        this.m_param[0].integer_value=(cmo_period<1 ? 9 : cmo_period);
        //--- период фактора сглаживания
        this.m_param[1].type=TYPE_UINT;
        this.m_param[1].integer_value=(ema_period<1 ? 12 : ema_period);
        //--- смещение индикатора по горизонтали
        this.m_param[2].type=TYPE_INT;
        this.m_param[2].integer_value=ma_shift;
        //--- тип цены или handle
        this.m_param[3].type=TYPE_UINT;
        this.m_param[3].integer_value=applied_price;
       }
     else
```

```
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
       //--- Создаём описание параметров.
       //--- Если символ или период графика не текущие, то к параметрам добавляются их о
       bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
       string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
       string param=StringFormat("(%s%s%lu,%lu)",symbol_period,(current ? "" : ":"),cmo_
       //--- Записываем описание параметров, наименование индикатора, его описание, заго
       this.SetParameters(param);
       this.SetName("VIDYA");
       this.SetDescription("Variable Index Dynamic Average");
       this.m_title=this.Name()+this.Parameters();
       this.m_category=IND_CATEGORY_TREND;
       this.m_digits=::Digits()+1;
       //--- Записываем описание буфера линии
       this.SetBufferDescription(0,this.m_title);
       //--- Записываем смещение в буфер 0
       this.SetBufferShift(0,ma_shift);
      }
  };
//+------------------------------------------------------------------+
//| Класс индикатора Volumes                                         |
//+------------------------------------------------------------------+
class CIndVolumes : public CIndMSTF
  {
public:
//--- Конструктор
   CIndVolumes(const string symbol,const ENUM_TIMEFRAMES timeframe,
               const ENUM_APPLIED_VOLUME applied_volume  // тип объема
              ) : CIndMSTF(IND_VOLUMES,1,symbol,timeframe)
     {
      //--- Устанавливаем размер массива параметров и заполняем его
      ::ResetLastError();
      if(::ArrayResize(this.m_param,1)==1)
        {
         ::ZeroMemory(this.m_param);
         //--- тип объема
         this.m_param[0].type=TYPE_UINT;
         this.m_param[0].integer_value=applied_volume;
        }
      else
         ::PrintFormat("%s: ArrayResize failed. Error %ld",__FUNCTION__,::GetLastError(
       //--- Создаём описание параметров.
       //--- Если символ или период графика не текущие, то к параметрам добавляются их о
       bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
       string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this.Ti
       string param=(current ? "" : StringFormat("(%s)",symbol_period));
       //--- Записываем описание параметров, наименование индикатора, его описание, заго
       this.SetParameters(param);
       this.SetName("Volumes");
       this.SetDescription("Volumes");
       this.m_title=this.Name()+this.Parameters();
       this.m_category=IND_CATEGORY_VOLUME;
       this.m_digits=0;
       //--- Записываем описание буфера линии
       this.SetBufferDescription(0,this.m_title);
      }
  };
```

```
//+------------------------------------------------------------------+
//| Класс пользовательского индикатора                               |
//+------------------------------------------------------------------+
class CIndCustom : public CIndMSTF
  {
public:
//--- Конструктор
   CIndCustom(const string symbol,const ENUM_TIMEFRAMES timeframe,
              const string path,                    // путь к индикатору (например, "
              const string name,                    // имя пользовательского индикато
              const uint   buffers,                 // количество буферов индикатора
              const MqlParam &param[]               // массив параметров пользователь
             ) : CIndMSTF(IND_CUSTOM,buffers,symbol,timeframe)
     {
      //--- Если передан пустой массив параметров - сообщаем об этом в журнал
      int total=(int)param.Size();
      if(total==0)
         ::PrintFormat("%s Error. Passed an empty array",__FUNCTION__);
      //--- Если массив не пустой и его размер увеличен на 1 (в первый параметр типа st
      ResetLastError();
      if(total>0 && ::ArrayResize(this.m_param,total+1)==total+1)
        {
         //--- Обнуляем данные в массиве и вписываем имя (путь к файлу и имя .ex5 файла
         ::ZeroMemory(this.m_param);
         //--- имя пользовательского индикатора
         this.m_param[0].type=TYPE_STRING;
         this.m_param[0].string_value=path;
         //--- заполняем массив параметров индикатора
         for(int i=0;i<total;i++)
           {
            this.m_param[i+1].type=param[i].type;
            this.m_param[i+1].double_value=param[i].double_value;
            this.m_param[i+1].integer_value=param[i].integer_value;
            this.m_param[i+1].string_value=param[i].string_value;
           }
         //--- Создаём описание параметров.
         //--- Если символ или период графика не текущие, то к параметрам добавляются и
         bool current=(this.Symbol()==::Symbol() && this.Timeframe()==::Period());
         string symbol_period=(current ? "" : ::StringFormat("%s,%s",this.Symbol(),this
         string param=(current ? "" : StringFormat("(%s)",symbol_period));
         //--- Записываем описание параметров, наименование индикатора, его описание, з
         this.SetParameters(param);
         this.SetName(name);
         this.SetDescription(name);
         this.m_title=this.Name()+this.Parameters();
         this.m_category=IND_CATEGORY_CUSTOM;
         //--- Записываем описание первого буфера линии
         this.SetBufferDescription(0,this.m_title);
        }
     }
  };
```

В представленном списке содержатся все классы для создания всех имеющихся в клиентском терминале технических индикаторов + класс для создания пользовательского индикатора. Все классы идентичны друг другу и основные моменты прокомментированы в коде классов.

Создание и работа не всех индикаторов, представленных выше, была протестирована. Тестировались только индикаторы на основе скользящих средних, рисующих одну линию на основном графике. В последующих статьях напишем шаблоны для создания мультисимвольных мультипериодных версий всех типов технических индикаторов.

По сути, уже всё готово для создания мульти- индикаторов. Но мы пойдём чуть дальше и создадим удобный инструмент для автоматизации создания и использования мультисимвольных мультипериодных индикаторов. Это будет класс-коллекция индикаторов, позволяющий легко создавать любые индикаторы на основе стандартных или пользовательских, превращая их в мультисимвольные мультипериодные индикаторы.

Класс-коллекция индикаторов

Класс-коллекция индикаторов по своей сути — обычный список указателей на объекты. Просто к нему же мы добавим методы для удобного создания индикаторов и добавления вновь созданных в коллекцию. Также будет возможность легко получить указатель на нужный индикатор и взять от него нужные данные. При добавлении нового индикатора в коллекцию, сначала будет проверено наличие точно такого же индикатора в коллекции. При его наличии новый созданный объект удаляется и возвращается указатель на уже существующий в коллекции — чтобы два разных объекта не ссылались на одну и ту же рассчётную часть.

Структура класса-коллекции будет повторять структуру класса базового мультисимвольного мультипериодного индикатора. С той лишь разницей, что большинство методов должны сначала найти требуемый индикатор по его хэндлу, а затем вызвать его нужный метод для установки или получения значения или результата действий.

В том же файле \MQL5\Include\IndMSTF\IndMSTF.mqh просто продолжим писать новый класс. Для его работы необходимо подключить файл класса динамического массива указателей на экземпляры класса CObject и его наследников CArrayObj:

```
//+------------------------------------------------------------------+
//| Класс-коллекция индикаторов                                      |
//+------------------------------------------------------------------+
#include <Arrays\ArrayObj.mqh>
class CMSTFIndicators
  {
private:

public:

  }
```

Впишем в тело класса объект списка и методы для работы с классом:

```
//+------------------------------------------------------------------+
//| Класс-коллекция индикаторов                                      |
//+------------------------------------------------------------------+
#include <Arrays\ArrayObj.mqh>
class CMSTFIndicators
  {
private:
   CArrayObj         m_list;
//--- Создаёт индикатор для переданного объекта
   bool              CreateIndicator(CIndMSTF *ind_obj);
//--- Добавляет указанный индикатор в коллекцию
   int               AddNewIndicator(CIndMSTF *ind_obj,const string source);
```

```
public:
//--- Возвращает (1) объект индикатора по хэндлу, (2) количество индикаторов в коллекци
   CIndMSTF         *GetIndicatorObj(const int ind_handle,const string source) const;
   uint              IndicatorsTotal(void)                  const { return this.m_list.

//--- Заполняет данными буферы (1) индикатора по хэндлу, (2) всех индикаторов в коллекц
   bool              Calculate(const int ind_handle);
   bool              Calculate(void);
//--- Устанавливает (1) указанное, (2) по умолчанию описание линии буфера индикатора
   void              SetPlotLabel(const uint plot_index,const string descript);
   void              SetPlotLabelFromBuffer(const uint plot_index,const int ind_handle,
//--- Устанавливает смещение указанному рисуемому буферу
   void              SetPlotShift(const uint plot_index,const int shift);
//--- (1) Устанавливает (2) возвращает инициализирующее значение указанного буфера указ
   void              SetBufferInitValue(const int ind_handle,const uint buffer_num,cons
   double            BufferInitValue(const int ind_handle,const uint buffer_num) const;

//--- Возвращает данные индикатора по хэндлу из указанного буфера по индексу (1) как ес
   double            GetData(const int ind_handle,const uint buffer_num,const uint inde
   double            GetDataTo(const string symbol_to,const ENUM_TIMEFRAMES timeframe_t

//--- (1) Копирует данные указанного буфера расчётной части указанного по хэндлу индика
//--- (2) возвращает количество данных в укакзанном буфере указанного по хэндлу индикат
   bool              DataToBuffer(const string symbol_to,const ENUM_TIMEFRAMES timefram
   uint              DataTotal(const int ind_handle,const uint buffer_num) const;

   //--- Возвращает (1) описание буфера, (2) состояние данных линии указанного буфера у
   //--- (3) состояние линии индикатора с учётом символа/периода графика, (4) состояние
   //--- (5)  состояние отношения линии индикатора с указанным уровнем с учётом символа
   string            BufferDescription(const int ind_handle,const uint buffer_num);
   ENUM_LINE_STATE   BufferLineState(const int ind_handle,const uint buffer_num,const i
   ENUM_LINE_STATE   BufferLineState(const string symbol,const ENUM_TIMEFRAMES timefram
   ENUM_LINE_STATE   BufferLineStateRelative(const int ind_handle,const int buffer_num,
   ENUM_LINE_STATE   BufferLineStateRelative(const string symbol,const ENUM_TIMEFRAMES
   string            CategoryDescription(const int ind_handle);

//--- Устанавливает (1) идентификатор, (2) Digits, (3) пользовательское описание, (4) о
   void              SetID(const int ind_handle,const int id);
   void              SetDigits(const int ind_handle,const int digits);
   void              SetDescription(const int ind_handle,const string descr);
   void              SetBufferDescription(const int ind_handle,const uint buffer_num,co

//--- Возвращает флаг серийности указанного буфера, (2) синхронизированности историческ
   bool              IsSeries(const int ind_handle,const uint buffer_num) const;
   bool              IsSynchronized(const int ind_handle) const;

//--- Возвращает (1) таймфрейм, (2) символ, (3) наименование, (4) список параметров, (5
//--- количество (7) буферов, (8) баров, (9) идентификатор, (10) описание, (11) заголов
//--- (13) количество параметрпов, (14) тип программы, описание (15) категории, (16) бу
   ENUM_TIMEFRAMES   Timeframe(const int ind_handle) const;
   string            Symbol(const int ind_handle) const;
   string            Name(const int ind_handle) const;
   string            Parameters(const int ind_handle) const;
   int               Digits(const int ind_handle) const;
   uint              BuffersTotal(const int ind_handle) const;
   uint              RatesTotal(const int ind_handle) const;
```

```
   int               ID(const int ind_handle) const;
   string            Description(const int ind_handle) const;
   string            Title(const int ind_handle) const;
   ENUM_IND_CATEGORY Category(const int ind_handle) const;
   uint              ParamsTotal(const int ind_handle) const;
//--- Возвращает (1) структуру параметров по индексу из массива, (2) описание таймфрейм
   MqlParam          GetMqlParam(const int ind_handle,const int index) const;
   string            TimeframeDescription(const int ind_handle)    const;
//--- Возвращает количество рассчитанных данных
   int               Calculated(const int ind_handle) const;

//--- Виртуальный метод, возвращающий тип объекта (индикатора)
      ENUM_INDICATOR    Type(const int ind_handle) const;

//--- Методы добавления индикаторов в коллекцию
   int               AddNewAC(const string symbol,const ENUM_TIMEFRAMES timeframe);
   int               AddNewAD(const string symbol,const ENUM_TIMEFRAMES timeframe,const
   int               AddNewADX(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
   int               AddNewADXWilder(const string symbol,const ENUM_TIMEFRAMES timefram
   int               AddNewAlligator(const string symbol,const ENUM_TIMEFRAMES timefram
                                                                                        
                                                                                        
                                                                                        
                                                                                        
                                                                                        
   int               AddNewAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
                                                                                  cons
                                                                                  cons
                                                                                  cons
                                                                                  cons
   int               AddNewAO(const string symbol,const ENUM_TIMEFRAMES timeframe);
   int               AddNewATR(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
   int               AddNewBearsPower(const string symbol,const ENUM_TIMEFRAMES timefra
   int               AddNewBullsPower(const string symbol,const ENUM_TIMEFRAMES timefra
   int               AddNewBands(const string symbol,const ENUM_TIMEFRAMES timeframe,co
                                                                                     co
                                                                                     co
                                                                                     co
   int               AddNewCCI(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
                                                                                  cons
   int               AddNewChaikin(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                                                                        
                                                                                        
   int               AddNewDEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,con
                                                                                     con
                                                                                     con
   int               AddNewDeMarker(const string symbol,const ENUM_TIMEFRAMES timeframe
   int               AddNewEnvelopes(const string symbol,const ENUM_TIMEFRAMES timefram
                                                                                        
                                                                                        
                                                                                        
   int               AddNewForce(const string symbol,const ENUM_TIMEFRAMES timeframe,co
                                                                                     co
```

```
                                                                                co
   int               AddNewFractals(const string symbol,const ENUM_TIMEFRAMES timeframe
   int               AddNewFrAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,co
                                                                                co
                                                                                co
   int               AddNewGator(const string symbol,const ENUM_TIMEFRAMES timeframe,co
                                                                                co
                                                                                co
                                                                                co
                                                                                co
                                                                                co
                                                                                co
                                                                                co
   int               AddNewIchimoku(const string symbol,const ENUM_TIMEFRAMES timeframe


   int               AddNewBWMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,co
   int               AddNewMomentum(const string symbol,const ENUM_TIMEFRAMES timeframe

   int               AddNewMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
                                                                                cons
   int               AddNewMA(const string symbol,const ENUM_TIMEFRAMES timeframe,const
                                                                              const
                                                                              const
                                                                              const
   int               AddNewOsMA(const string symbol,const ENUM_TIMEFRAMES timeframe,con
                                                                                con
                                                                                con
                                                                                con
   int               AddNewMACD(const string symbol,const ENUM_TIMEFRAMES timeframe,con
                                                                                con
                                                                                con
                                                                                con
   int               AddNewOBV(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
   int               AddNewSAR(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
                                                                                cons
   int               AddNewRSI(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
                                                                                cons
   int               AddNewRVI(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
   int               AddNewStdDev(const string symbol,const ENUM_TIMEFRAMES timeframe,c
                                                                                  c
                                                                                  c
                                                                                  c
   int               AddNewStochastic(const string symbol,const ENUM_TIMEFRAMES timefra




   int               AddNewTEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,con
                                                                                con
                                                                                con
   int               AddNewTriX(const string symbol,const ENUM_TIMEFRAMES timeframe,con
                                                                                con
   int               AddNewWPR(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
   int               AddNewVIDyA(const string symbol,const ENUM_TIMEFRAMES timeframe,co
                                                                                co
                                                                                co
```

```
                                                                                                  cc
   int               AddNewVolumes(const string symbol,const ENUM_TIMEFRAMES timeframe,
   int               AddNewCustom(const string symbol,const ENUM_TIMEFRAMES timeframe,c
                                                                                                  c
                                                                                                  c
                                                                                                  c
//--- Таймер
   void OnTimer(void)
     {
      //--- В цикле по всем индикаторам коллекции
      int total=this.m_list.Total();
      for(int i=0;i<total;i++)
        {
         //--- получаем указатель на очередной объект-индикатор
         //--- и вызываем его таймер
         CIndMSTF *obj=this.m_list.At(i);
         if(obj!=NULL)
            obj.OnTimer();
        }
     }
//--- Конструктор/Деструктор
                     CMSTFIndicators(void){ this.m_list.Clear(); }
                    ~CMSTFIndicators(void){;}
  };
```

Реализация методов для работы с индикаторами в списке:

```
//+------------------------------------------------------------------+
//| Создаёт расчётную часть индикатора для переданного объекта       |
//+------------------------------------------------------------------+
bool CMSTFIndicators::CreateIndicator(CIndMSTF *ind_obj)
  {
   //--- Если расчётную часть индикатора создать не удалось
   if(!ind_obj.CreateIndicator())
     {
      //--- ищем индекс объекта-индикатора в списке коллекции
      //--- и по полученному индексу удаляем объект-индикатор из списка коллекции
      this.m_list.Sort();
      int index=this.m_list.Search(ind_obj);
      this.m_list.Delete(index);
      //--- Возвращаем false
      return false;
     }
//--- Расчётная часть успешно создана - возвращаем true
   return true;
  }
//+------------------------------------------------------------------+
//| Возвращает объект индикатора по хэндлу расчётной части           |
//+------------------------------------------------------------------+
CIndMSTF *CMSTFIndicators::GetIndicatorObj(const int ind_handle,const string source) cc
  {
//--- Если в метод передан невалидный хэндл - сообщаем об это и возвращаем NULL
   if(ind_handle==INVALID_HANDLE)
     {
      ::PrintFormat("%s: Error handle",source);
```

```
      return NULL;
     }
//--- В цикле по всем объектам-индикаторам в списке коллекции
   int total=this.m_list.Total();
   for(int i=0;i<total;i++)
     {
      //--- получаем указатель на очередной объект-индикатор
      CIndMSTF *obj=this.m_list.At(i);
      if(obj==NULL)
         continue;
      //--- Если хэндл индикатора равен переданному в метод -
      //--- возвращаем указатель на найденный объект-индикатор
      if(obj.Handle()==ind_handle)
         return obj;
     }
//--- Ничего не нашли - возвращаем NULL
   return NULL;
  }
//+------------------------------------------------------------------+
//| Заполняет данными буферы индикатора по хэндлу                    |
//+------------------------------------------------------------------+
bool CMSTFIndicators::Calculate(const int ind_handle)
  {
   //--- Получаем по хэндлу указатель на объект-индикатор
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
      return false;
   //--- Возвращаем результат работы метода Calculate полученного по хэндлу объекта-ин
   return obj.Calculate();
  }
//+------------------------------------------------------------------+
//| Заполняет данными буферы всех индикаторов в коллекции            |
//+------------------------------------------------------------------+
bool CMSTFIndicators::Calculate(void)
  {
   //--- Объявляем переменную для хранения результата
   bool res=true;
//--- В цикле по всем объектам-индикаторам в списке коллекции
   int total=this.m_list.Total();
   for(int i=0;i<total;i++)
     {
      //--- получаем указатель на очередной объект-индикатор
      CIndMSTF *obj=this.m_list.At(i);
      if(obj==NULL)
         continue;
      //--- Добавляем к переменной res результат вызова метода Calculate очередного объ
      res &=obj.Calculate();
      //--- Если метод отработал с ошибкой - сообщаем об этом в журнал
      if(!res)
         ::PrintFormat("%s::%s: Error in indicator calculation: %s",__FUNCTION__,obj.Ti
     }
//--- Если общий результат false - сообщаем об этом в журнал
   if(!res)
      ::PrintFormat("%s: Not all indicators have been calculated successfully. It is ne
//--- Возвращаем результат вызова методов Calculate всех индикаторов в коллекции
   return res;
  }
```

```
//+------------------------------------------------------------------+
//| Возвращает данные индикатора по хэндлу                           |
//| из указанного буфера по индексу как есть                         |
//+------------------------------------------------------------------+
double CMSTFIndicators::GetData(const int ind_handle,const uint buffer_num,const uint i
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return EMPTY_VALUE;
     }
//--- Возвращаем данные из указанного буфера индикатора по индексу, переданному в метод
   return obj.GetData(buffer_num,index);
  }
//+------------------------------------------------------------------+
//| Возвращает данные индикатора по хэндлу                           |
//| из указанного буфера по индексу на указанный символ/таймфрейм    |
//+------------------------------------------------------------------+
double CMSTFIndicators::GetDataTo(const string symbol_to,const ENUM_TIMEFRAMES timefram
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return EMPTY_VALUE;
     }
//--- Возвращаем данные из указанного буфера индикатора по индексу, переданному в метод
   return obj.GetDataTo(symbol_to,timeframe_to,buffer_num,index);
  }
//+------------------------------------------------------------------+
//| Заполняет переданный буфер индикатора данными                    |
//+------------------------------------------------------------------+
bool CMSTFIndicators::DataToBuffer(const string symbol_to,const ENUM_TIMEFRAMES timefra
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return false;
     }
//--- Заполняем переданный в метод массив-буфер из указанного буфера индикатора
   return obj.DataToBuffer(symbol_to,timeframe_to,buffer_num,limit,buffer);
  }
//+------------------------------------------------------------------+
//| Устанавливает указанное описание для линии буфера                |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetPlotLabel(const uint plot_index,const string descript)
  {
   ::PlotIndexSetString(plot_index,PLOT_LABEL,descript);
  }
//+------------------------------------------------------------------+
//| Устанавливает описание по умолчанию линии буфера                 |
//+------------------------------------------------------------------+
```

```
void CMSTFIndicators::SetPlotLabelFromBuffer(const uint plot_index,const int ind_handle
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем в указанный рисуемый буфер описание указанного буфера индикатора
   ::PlotIndexSetString(plot_index,PLOT_LABEL,obj.BufferDescription(buffer_num));
  }
//+------------------------------------------------------------------+
//| Устанавливает смещение указанному рисуемому буферу               |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetPlotShift(const uint plot_index,const int shift)
  {
   ::PlotIndexSetInteger(plot_index,PLOT_SHIFT,shift);
  }
//+------------------------------------------------------------------+
//| Возвращает описание указанного буфера                            |
//| указанного по хэндлу индикатора                                  |
//+------------------------------------------------------------------+
string CMSTFIndicators::BufferDescription(const int ind_handle,const uint buffer_num)
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если указатель на объект получен - возвращаем из него описание указанного буфера.
   return(obj!=NULL ? obj.BufferDescription(buffer_num) : ::StringFormat("%s: Failed to
  }
//+------------------------------------------------------------------+
//| Устанавливает инициализирующее значение указанного буфера        |
//| указанного по хэндлу индикатора                                  |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetBufferInitValue(const int ind_handle,const uint buffer_num,con
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем для указанного буфера указанное инициализирующее "пустое" значение
   obj.SetBufferInitValue(buffer_num,value);
  }
//+------------------------------------------------------------------+
//| Возвращает инициализирующее значение указанного буфера           |
//| указанного по хэндлу индикатора                                  |
//+------------------------------------------------------------------+
double CMSTFIndicators::BufferInitValue(const int ind_handle,const uint buffer_num) con
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
```

```
      return WRONG_VALUE;
     }
//--- Возвращаем инициализирующее "пустое" значение, установленное для указанного буфера
   return obj.BufferInitValue(buffer_num);
  }
//+------------------------------------------------------------------+
//| Возвращает состояние данных линии указанного буфера              |
//| указанного по хэндлу индикатора на указанном баре                |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CMSTFIndicators::BufferLineState(const int ind_handle,const uint buffer
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return LINE_STATE_NONE;
     }
//--- Возвращаем состояние линии указанного буфера на указанном индексе
   return obj.BufferLineState(buffer_num,index);
  }
//+------------------------------------------------------------------+
//| Возвращает состояние данных линии указанного буфера              |
//| указанного по хэндлу индикатора на баре символа/таймфрейма       |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CMSTFIndicators::BufferLineState(const string symbol,const ENUM_TIMEFRA
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return LINE_STATE_NONE;
     }
//--- Получаем время бара, переданного в метод
   datetime array[1];
   if(::CopyTime(symbol,timeframe,index,1,array)!=1)
     {
      ::PrintFormat("%s::%s: Failed to get the time of the bar with index %ld. Error %l
      return LINE_STATE_NONE;
     }
//--- Получаем номер бара в буфере объекта-индикатора, соответствующий найденному времен
   int bar=::iBarShift(obj.Symbol(),obj.Timeframe(),array[0]);
//--- Если бар получен - возвращаем состояние линии на найденном баре, иначе - неопреде
   return(bar!=WRONG_VALUE ? obj.BufferLineState(buffer_num,bar) : LINE_STATE_NONE);
  }
//+------------------------------------------------------------------+
//| Возвращает соотношение данных линии указанного буфера            |
//| указанного по хэндлу индикатора на указанном баре                |
//| с указанными значениями                                          |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CMSTFIndicators::BufferLineStateRelative(const int ind_handle,const int
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
```

```
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return LINE_STATE_NONE;
     }
//--- Возвращаем соотношение линии индикатора и уровня в указанном буфере на указанном
   return obj.BufferLineStateRelative(buffer_num,index,level0,level1);
  }
//+------------------------------------------------------------------+
//| Возвращает соотношение данных линии указанного буфера            |
//| указанного по хэндлу индикатора на указанном баре                |
//| с указанными значениями на указанном символе/периоде графика    |
//+------------------------------------------------------------------+
ENUM_LINE_STATE CMSTFIndicators::BufferLineStateRelative(const string symbol,const ENUM
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return LINE_STATE_NONE;
     }
//--- Возвращаем соотношение линии индикатора и уровня в указанном буфере на указанном
   return obj.BufferLineStateRelative(symbol,timeframe,buffer_num,index,level0,level1);
  }
//+------------------------------------------------------------------+
//| Возвращает описание категории                                    |
//+------------------------------------------------------------------+
string CMSTFIndicators::CategoryDescription(const int ind_handle)
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем описание категории. Иначе - текст ошибки
   return(obj!=NULL ? obj.CategoryDescription() : ::StringFormat("%s: Failed to get ind
  }
//+------------------------------------------------------------------+
//| Устанавливает идентификатор                                      |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetID(const int ind_handle,const int id)
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем идентификатор для полученного объекта
   obj.SetID(id);
  }
//+------------------------------------------------------------------+
//| Устанавливает Digits индикатора                                  |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetDigits(const int ind_handle,const int digits)
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
```

```
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем Digits для полученного объекта
   obj.SetDigits(digits);
  }
//+------------------------------------------------------------------+
//| Устанавливает пользовательское описание                          |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetDescription(const int ind_handle,const string descr)
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем описание для полученного объекта
   obj.SetDescription(descr);
  }
//+------------------------------------------------------------------+
//| Устанавливает описание указанного буфера                         |
//+------------------------------------------------------------------+
void CMSTFIndicators::SetBufferDescription(const int ind_handle,const uint buffer_num,c
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return;
     }
//--- Устанавливаем описание для указанного буфера полученного объекта
   obj.SetBufferDescription(buffer_num,descr);
  }
//+------------------------------------------------------------------+
//| Возвращает флаг серийности указанного буфера                     |
//+------------------------------------------------------------------+
bool CMSTFIndicators::IsSeries(const int ind_handle,const uint buffer_num) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return false;
     }
//--- Возвращаем флаг серийности указанного буфера полученного объекта
   return obj.IsSeries(buffer_num);
  }
//+------------------------------------------------------------------+
//| Возвращает флаг синхронизированности                             |
//| исторических данных по символу/периоду                           |
//+------------------------------------------------------------------+
bool CMSTFIndicators::IsSynchronized(const int ind_handle) const
  {
```

```
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return false;
     }
//--- Возвращаем флаг синхронизированности полученного объекта
   return obj.IsSynchronized();
  }
//+------------------------------------------------------------------+
//| Возвращает таймфрейм указанного индикатора                       |
//+------------------------------------------------------------------+
ENUM_TIMEFRAMES CMSTFIndicators::Timeframe(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return WRONG_VALUE;
     }
//--- Возвращаем таймфрейм полученного объекта
   return obj.Timeframe();
  }
//+------------------------------------------------------------------+
//| Возвращает символ указанного индикатора                          |
//+------------------------------------------------------------------+
string CMSTFIndicators::Symbol(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем наименование символа. Иначе - текст ошибки
   return(obj!=NULL ? obj.Symbol() : ::StringFormat("%s: Failed to get indicator object
  }
//+------------------------------------------------------------------+
//| Возвращает наименование указанного индикатора                    |
//+------------------------------------------------------------------+
string CMSTFIndicators::Name(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем наименование индикатора. Иначе - текст ошибки
   return(obj!=NULL ? obj.Name() : ::StringFormat("%s: Failed to get indicator object",
  }
//+------------------------------------------------------------------+
//| Возвращает список параметров указанного индикатора               |
//+------------------------------------------------------------------+
string CMSTFIndicators::Parameters(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем список параметров индикатора. Иначе - текст ошибк
   return(obj!=NULL ? obj.Parameters() : ::StringFormat("%s: Failed to get indicator ob
  }
//+------------------------------------------------------------------+
//| Возвращает Digits указанного индикатора                          |
```

```
//+------------------------------------------------------------------+
int CMSTFIndicators::Digits(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return WRONG_VALUE;
     }
//--- Возвращаем Digits полученного объекта
   return obj.Digits();
  }
//+------------------------------------------------------------------+
//| Возвращает количество буферов указанного индикатора              |
//+------------------------------------------------------------------+
uint CMSTFIndicators::BuffersTotal(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return 0;
     }
//--- Возвращаем количество буферов полученного объекта
   return obj.BuffersTotal();
  }
//+------------------------------------------------------------------+
//| Возвращает количество баров таймсерии для указанного индикатора  |
//+------------------------------------------------------------------+
uint CMSTFIndicators::RatesTotal(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return 0;
     }
//--- Возвращаем количество баров таймсерии полученного объекта
   return obj.RatesTotal();
  }
//+------------------------------------------------------------------+
//| Возвращает идентификатор указанного индикатора                   |
//+------------------------------------------------------------------+
int CMSTFIndicators::ID(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return WRONG_VALUE;
     }
//--- Возвращаем идентификатор полученного объекта
   return obj.ID();
```

```
  }
//+------------------------------------------------------------------+
//| Возвращает описание указанного индикатора                        |
//+------------------------------------------------------------------+
string CMSTFIndicators::Description(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем описание индикатора. Иначе - текст ошибки
   return(obj!=NULL ? obj.Description() : ::StringFormat("%s: Failed to get indicator o
  }
//+------------------------------------------------------------------+
//| Возвращает заголовок указанного индикатора                       |
//+------------------------------------------------------------------+
string CMSTFIndicators::Title(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем заголовок индикатора. Иначе - текст ошибки
   return(obj!=NULL ? obj.Title() : ::StringFormat("%s: Failed to get indicator object"
  }
//+------------------------------------------------------------------+
//| Возвращает категорию указанного индикатора                       |
//+------------------------------------------------------------------+
ENUM_IND_CATEGORY CMSTFIndicators::Category(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return IND_CATEGORY_NONE;
     }
//--- Возвращаем категорию полученного объекта
   return obj.Category();
  }
//+------------------------------------------------------------------+
//| Возвращает количество параметров указанного индикатора           |
//+------------------------------------------------------------------+
uint CMSTFIndicators::ParamsTotal(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return 0;
     }
//--- Возвращаем количество параметров полученного объекта
   return obj.ParamsTotal();
  }
//+------------------------------------------------------------------+
//| Возвращает структуру параметров по индексу из массива            |
//| для указанного индикатора                                        |
//+------------------------------------------------------------------+
MqlParam CMSTFIndicators::GetMqlParam(const int ind_handle,const int index) const
  {
```

```
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      MqlParam null;
      ::ZeroMemory(null);
      return null;
     }
//--- Возвращаем структуру параметров полученного объекта по индексу из массива парамет
   return obj.GetMqlParam(index);
  }
//+------------------------------------------------------------------+
//| Возвращает описание таймфрейма для указанного индикатора         |
//+------------------------------------------------------------------+
string CMSTFIndicators::TimeframeDescription(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
//--- Если объект получен, возвращаем описание таймфрейма индикатора. Иначе - текст оши
   return(obj!=NULL ? obj.Description() : ::StringFormat("%s: Failed to get indicator o
  }
//+------------------------------------------------------------------+
//| Возвращает количество рассчитанных данных указанного индикатора  |
//+------------------------------------------------------------------+
int CMSTFIndicators::Calculated(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return WRONG_VALUE;
     }
//--- Возвращаем количество рассчитанных данных полученного объекта
   return obj.Calculated();
  }
//+------------------------------------------------------------------+
//| Возвращает тип указанного индикатора                             |
//+------------------------------------------------------------------+
ENUM_INDICATOR CMSTFIndicators::Type(const int ind_handle) const
  {
//--- Получаем указатель на объект-индикатор по хэндлу, переданному в метод
   CIndMSTF *obj=this.GetIndicatorObj(ind_handle,__FUNCTION__);
   if(obj==NULL)
     {
      ::PrintFormat("%s: Failed to get indicator object",__FUNCTION__);
      return (ENUM_INDICATOR)WRONG_VALUE;
     }
//--- Возвращаем тип индикатора полученного объекта
   return (ENUM_INDICATOR)obj.Type();
  }
```

Логика всех методов прокомментирована в листинге методов. Сначала получаем указатель на требуемый инедикатор в списке, затем устанавливаем или возвращаем его свойство, либо производим расчёт и возвращаем его результат. Все вызываемые методы рассматривались выше при обсуждении базового класса мультисимвольного мультипериодного индикатора.

Реализация методов для создания новых объектов-индикаторов:

```
//+------------------------------------------------------------------+
//| Добавляет указанный индикатор в коллекцию                        |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewIndicator(CIndMSTF *ind_obj,const string source)
  {
//--- Устанавливаем списку коллекции флаг сортированного списка
   this.m_list.Sort();
//--- Ищем в списке индекс аналогичного переданному в метод объекта-индикатора
   int index=this.m_list.Search(ind_obj);
//--- Если такой индикатор с такими же параметрами уже есть в списке,
   if(index>WRONG_VALUE)
     {
      //--- сообщаем об этом в журнал и удаляем новый объект-индикатор
      ::PrintFormat("%s: The %s indicator with such parameters %s is already in the col
      delete ind_obj;
      //--- Получаем указатель на уже существующий объект-индикатор в списке и возвраща
      ind_obj=this.m_list.At(index);
      return(ind_obj!=NULL ? ind_obj.Handle() : INVALID_HANDLE);
     }
//--- Если такого индикатора нет в списке, но его не удалось поместить в список
   if(!this.m_list.Add(ind_obj))
     {
      //--- сообщаем об ошибке в журнал, удаляем объект-индикатор и возвращаем INVALID_
      ::PrintFormat("%s: Error. Failed to add %s indicator to collection",source,ind_ob
      delete ind_obj;
      return INVALID_HANDLE;
     }
//--- Если индикатор помещён в список, но для него не удалось создать расчётную часть -
//--- (при ошибке создания расчётной части из списка объект-индикатор удаляется в метод
   if(!this.CreateIndicator(ind_obj))
      return INVALID_HANDLE;
//--- Всё успешно - сообщаем о добавлении нового индикатора в коллекцию и возвращаем ег
   ::PrintFormat("%s: %s indicator (handle %ld) added to the collection",source,ind_obj
   return ind_obj.Handle();
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Accelerator Oscillator           |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewAC(const string symbol,const ENUM_TIMEFRAMES timeframe)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndAC *ind_obj=new CIndAC(symbol,timeframe);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create AC indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Accumulation/Distribution        |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewAD(const string symbol,const ENUM_TIMEFRAMES timeframe,const
  {
```

```
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndAD *ind_obj=new CIndAD(symbol,timeframe,applied_volume);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create A/D indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Average Directional Movement Index |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewADX(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndADX *ind_obj=new CIndADX(symbol,timeframe,adx_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create ADX indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор                                  |
//| Average Directional Movement Index by Welles Wilder              |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewADXWilder(const string symbol,const ENUM_TIMEFRAMES timefran
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndADXW *ind_obj=new CIndADXW(symbol,timeframe,adx_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create ADX Wilder indicator object",__FUNCTIO
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Alligator                        |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewAlligator(const string symbol,const ENUM_TIMEFRAMES timefran
                                     const int jaw_period=13,
                                     const int jaw_shift=8,
                                     const int teeth_period=8,
                                     const int teeth_shift=5,
                                     const int lips_period=5,
                                     const int lips_shift=3,
                                     const ENUM_MA_METHOD ma_method=MODE_SMMA,
                                     const ENUM_APPLIED_PRICE applied_price=PRICE_MEDI
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndAlligator *ind_obj=new CIndAlligator(symbol,timeframe,jaw_period,jaw_shift,teeth
   if(ind_obj==NULL)
```

```
      {
       ::PrintFormat("%s: Error. Failed to create Alligator indicator object",__FUNCTION
       return INVALID_HANDLE;
      }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Adaptive Moving Average          |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                               const int ama_period=9,
                               const int fast_ma_period=2,
                               const int slow_ma_period=30,
                               const int ama_shift=0,
                               const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndAMA *ind_obj=new CIndAMA(symbol,timeframe,ama_period,fast_ma_period,slow_ma_peri
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create AMA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Awesome Oscillator               |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewAO(const string symbol,const ENUM_TIMEFRAMES timeframe)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndAO *ind_obj=new CIndAO(symbol,timeframe);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create AO indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Average True Range               |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewATR(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndATR *ind_obj=new CIndATR(symbol,timeframe,ma_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create ATR indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
```

```
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Bears Power                      |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewBearsPower(const string symbol,const ENUM_TIMEFRAMES timefra
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndBears *ind_obj=new CIndBears(symbol,timeframe,ma_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Bears indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Bulls Power                      |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewBullsPower(const string symbol,const ENUM_TIMEFRAMES timefra
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndBulls *ind_obj=new CIndBulls(symbol,timeframe,ma_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Bulls indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Bollinger Bands®                 |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewBands(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                 const int bands_period=20,
                                 const int bands_shift=0,
                                 const double deviation=2.0,
                                 const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndBands *ind_obj=new CIndBands(symbol,timeframe,bands_period,bands_shift,deviation
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Bands indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Commodity Channel Index          |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewCCI(const string symbol,const ENUM_TIMEFRAMES timeframe,
                               const int ma_period=14,
                               const ENUM_APPLIED_PRICE applied_price=PRICE_TYPICAL)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
```

```
   CIndCCI *ind_obj=new CIndCCI(symbol,timeframe,ma_period,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create CCI indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Chaikin Oscillator               |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewChaikin(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                   const int fast_ma_period=3,
                                   const int slow_ma_period=10,
                                   const ENUM_MA_METHOD ma_method=MODE_EMA,
                                   const ENUM_APPLIED_VOLUME applied_volume=VOLUME_TIC
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndCHO *ind_obj=new CIndCHO(symbol,timeframe,fast_ma_period,slow_ma_period,ma_metho
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Chaikin indicator object",__FUNCTION__
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Double Exponential Moving Average |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewDEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                const int ma_period=14,
                                const int ma_shift=0,
                                const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndDEMA *ind_obj=new CIndDEMA(symbol,timeframe,ma_period,ma_shift,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create DEMA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор DeMarker                         |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewDeMarker(const string symbol,const ENUM_TIMEFRAMES timeframe
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndDeM *ind_obj=new CIndDeM(symbol,timeframe,ma_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create DeMarker indicator object",__FUNCTION_
      return INVALID_HANDLE;
```

```
      }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Envelopes                        |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewEnvelopes(const string symbol,const ENUM_TIMEFRAMES timefram
                                     const int ma_period=14,
                                     const int ma_shift=0,
                                     const ENUM_MA_METHOD ma_method=MODE_SMA,
                                     const ENUM_APPLIED_PRICE applied_price=PRICE_CLOS
                                     const double deviation=0.1)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndEnvelopes *ind_obj=new CIndEnvelopes(symbol,timeframe,ma_method,ma_shift,ma_meth
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Envelopes indicator object",__FUNCTION
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Force Index                      |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewForce(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                 const int ma_period=13,
                                 const ENUM_MA_METHOD ma_method=MODE_SMA,
                                 const ENUM_APPLIED_VOLUME applied_volume=VOLUME_TICK)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndForce *ind_obj=new CIndForce(symbol,timeframe,ma_period,ma_method,applied_volume
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Force indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Fractals                         |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewFractals(const string symbol,const ENUM_TIMEFRAMES timeframe
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndFractals *ind_obj=new CIndFractals(symbol,timeframe);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Fractals indicator object",__FUNCTION_
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
```

```
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Fractal Adaptive Moving Average  |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewFrAMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                 const int ma_period=14,
                                 const int ma_shift=0,
                                 const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndFrAMA *ind_obj=new CIndFrAMA(symbol,timeframe,ma_period,ma_shift,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create FrAMA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Gator                            |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewGator(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                 const int jaw_period=13,
                                 const int jaw_shift=8,
                                 const int teeth_period=8,
                                 const int teeth_shift=5,
                                 const int lips_period=5,
                                 const int lips_shift=3,
                                 const ENUM_MA_METHOD ma_method=MODE_SMMA,
                                 const ENUM_APPLIED_PRICE applied_price=PRICE_MEDIAN)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndGator *ind_obj=new CIndGator(symbol,timeframe,jaw_period,jaw_shift,teeth_period,
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Gator indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Ichimoku Kinko Hyo               |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewIchimoku(const string symbol,const ENUM_TIMEFRAMES timeframe
                                    const int tenkan_sen=9,
                                    const int kijun_sen=26,
                                    const int senkou_span_b=52)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndIchimoku *ind_obj=new CIndIchimoku(symbol,timeframe,tenkan_sen,kijun_sen,senkou_
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Ichimoku indicator object",__FUNCTION_
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
```

```
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Market Facilitation Index        |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewBWMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,co
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndBWMFI *ind_obj=new CIndBWMFI(symbol,timeframe,applied_volume);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create BW MFI indicator object",__FUNCTION__)
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Momentum                         |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewMomentum(const string symbol,const ENUM_TIMEFRAMES timeframe
                                    const int mom_period=14,
                                    const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndMomentum *ind_obj=new CIndMomentum(symbol,timeframe,mom_period,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Momentum indicator object",__FUNCTION_
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Money Flow Index                 |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewMFI(const string symbol,const ENUM_TIMEFRAMES timeframe,
                               const int ma_period=14,
                               const ENUM_APPLIED_VOLUME applied_volume=VOLUME_TICK)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndMFI *ind_obj=new CIndMFI(symbol,timeframe,ma_period,applied_volume);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create MFI indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Moving Average                   |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                              const int ma_period=10,
                              const int ma_shift=0,
```

```
                                 const ENUM_MA_METHOD ma_method=MODE_SMA,
                                 const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndMA *ind_obj=new CIndMA(symbol,timeframe,ma_period,ma_shift,ma_method,applied_pri
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create MA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Moving Average of Oscillator     |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewOsMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                const int fast_ema_period=12,
                                const int slow_ema_period=26,
                                const int signal_period=9,
                                const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndOsMA *ind_obj=new CIndOsMA(symbol,timeframe,fast_ema_period,slow_ema_period,sign
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create OsMA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор                                  |
//| Moving Averages Convergence/Divergence                           |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewMACD(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                const int fast_ema_period=12,
                                const int slow_ema_period=26,
                                const int signal_period=9,
                                const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndMACD *ind_obj=new CIndMACD(symbol,timeframe,fast_ema_period,slow_ema_period,sign
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create MACD indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор On Balance Volume                |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewOBV(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
  {
```

```
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndOBV *ind_obj=new CIndOBV(symbol,timeframe,applied_volume);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create OBV indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Parabolic Stop and Reverse system |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewSAR(const string symbol,const ENUM_TIMEFRAMES timeframe,
                               const double step=0.02,
                               const double maximum=0.2)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndSAR *ind_obj=new CIndSAR(symbol,timeframe,step,maximum);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create SAR indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Relative Strength Index          |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewRSI(const string symbol,const ENUM_TIMEFRAMES timeframe,
                               const int ma_period=14,
                               const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndRSI *ind_obj=new CIndRSI(symbol,timeframe,ma_period,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create RSI indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Relative Vigor Index             |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewRVI(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndRVI *ind_obj=new CIndRVI(symbol,timeframe,ma_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create RVI indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
```

```
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Standard Deviation               |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewStdDev(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                  const int ma_period=20,
                                  const int ma_shift=0,
                                  const ENUM_MA_METHOD ma_method=MODE_SMA,
                                  const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndStdDev *ind_obj=new CIndStdDev(symbol,timeframe,ma_period,ma_shift,ma_method,app
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create StdDev indicator object",__FUNCTION__)
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Stochastic Oscillator            |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewStochastic(const string symbol,const ENUM_TIMEFRAMES timefra
                                      const int Kperiod=5,
                                      const int Dperiod=3,
                                      const int slowing=3,
                                      const ENUM_MA_METHOD ma_method=MODE_SMA,
                                      const ENUM_STO_PRICE price_field=STO_LOWHIGH)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndStoch *ind_obj=new CIndStoch(symbol,timeframe,Kperiod,Dperiod,slowing,ma_method,
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Stochastic indicator object",__FUNCTIO
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Triple Exponential Moving Average |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewTEMA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                const int ma_period=14,
                                const int ma_shift=0,
                                const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndTEMA *ind_obj=new CIndTEMA(symbol,timeframe,ma_period,ma_shift,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create TEMA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
```

```
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }

//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор                                  |
//| Triple Exponential Moving Averages Oscillator                    |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewTriX(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                const int ma_period=14,
                                const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndTriX *ind_obj=new CIndTriX(symbol,timeframe,ma_period,applied_price);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create TriX indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Larry Williams' Percent Range    |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewWPR(const string symbol,const ENUM_TIMEFRAMES timeframe,cons
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndWPR *ind_obj=new CIndWPR(symbol,timeframe,calc_period);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create WPR indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Variable Index Dynamic Average   |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewVIDyA(const string symbol,const ENUM_TIMEFRAMES timeframe,
                                 const int cmo_period=9,
                                 const int ema_period=12,
                                 const int ma_shift=0,
                                 const ENUM_APPLIED_PRICE applied_price=PRICE_CLOSE)
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndVIDyA *ind_obj=new CIndVIDyA(symbol,timeframe,cmo_period,ema_period,ma_shift,app
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create VIDyA indicator object",__FUNCTION__);
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию индикатор Volumes                          |
```

```
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewVolumes(const string symbol,const ENUM_TIMEFRAMES timeframe,
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndVolumes *ind_obj=new CIndVolumes(symbol,timeframe,applied_volume);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create Volumes indicator object",__FUNCTION__
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
//+------------------------------------------------------------------+
//| Добавляет в коллекцию пользовательский индикатор                 |
//+------------------------------------------------------------------+
int CMSTFIndicators::AddNewCustom(const string symbol,const ENUM_TIMEFRAMES timeframe,c
  {
//--- Создаём новый объект индикатора. При ошибке сообщаем об этом в журнал и возвращае
   CIndCustom *ind_obj=new CIndCustom(symbol,timeframe,path,name,buffers,param);
   if(ind_obj==NULL)
     {
      ::PrintFormat("%s: Error. Failed to create %s custom indicator object",__FUNCTION
      return INVALID_HANDLE;
     }
//--- Возвращаем результат добавления созданного объекта-индикатора в список коллекцию
   return this.AddNewIndicator(ind_obj,__FUNCTION__);
  }
```

Все коды для класса коллекции индикаторов готовы. В каждом методе его логика прокомментирована, и самостоятельно разобраться в ней не составит труда.

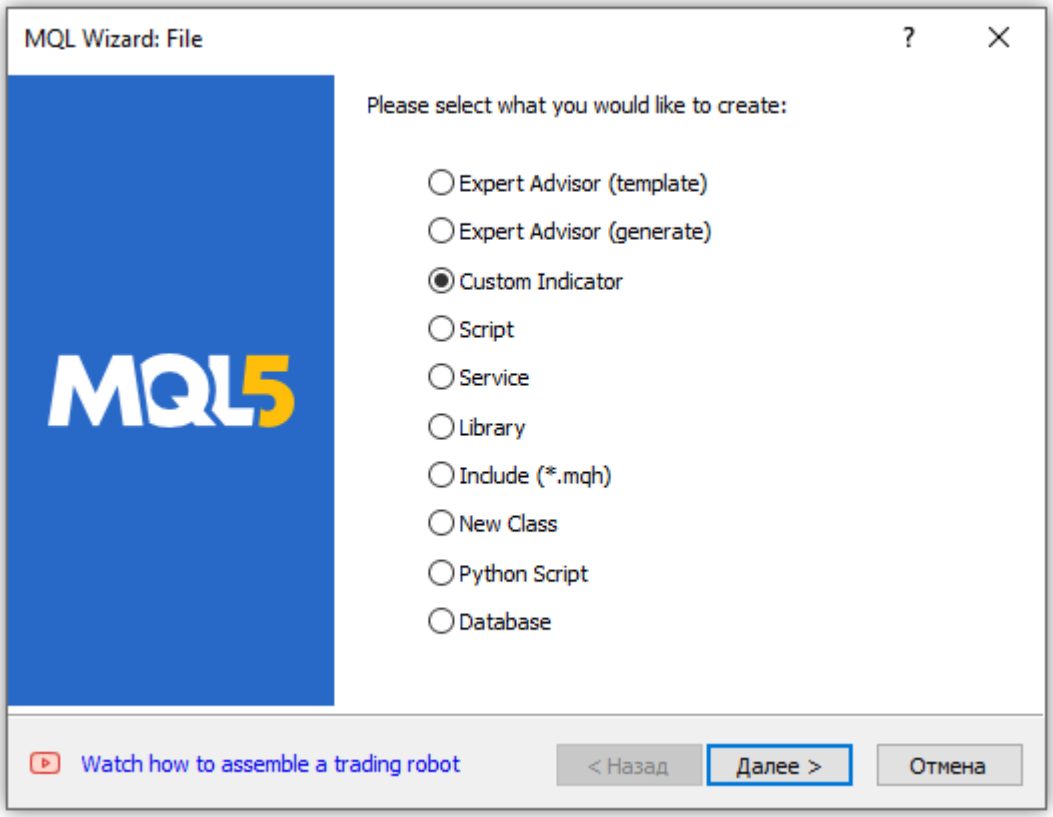
Тестирование

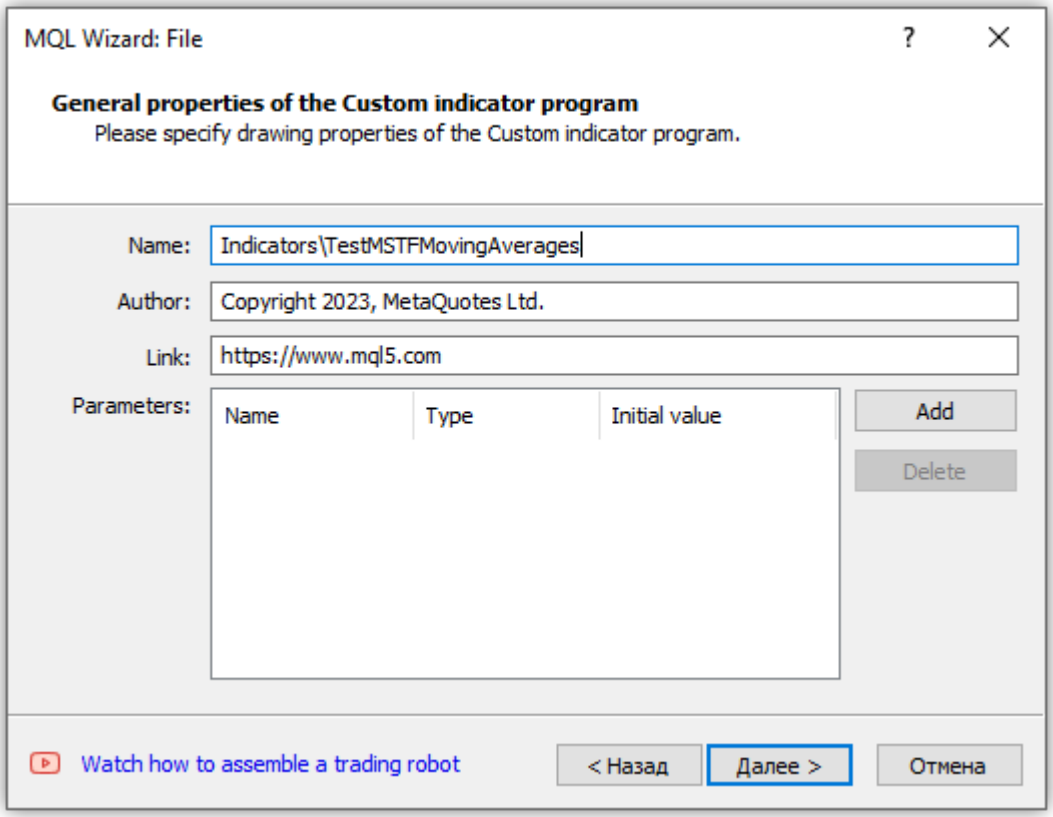
Для тестирования работы с классами мульти- индикаторов и их коллекцией создадим простой индикатор, в котором организуем выбор из нескольких скользящих средних. Цель сегодняшних тестов — протестировать работу классов с индикаторами, рисующими одну линию из одного буфера в главном окне графика. Для этой цели хорошо подойдут скользящие средние из набора трендовых индикаторов клиентского терминала. В последующих статьях создадим шаблоны для быстрого создания и работы с любыми индикаторами из набора индикаторов в терминале + работа с пользовательскими индикаторами. И постепенно полностью доработаем созданные сегодня классы для правильной работы с любыми типами индикаторов.

Тестовый индикатор будет выводить линии созданных мульти- индикаторов на график, а их данные — на информационную панель, создание которой было описано в первой статье цикла работы с индикаторами.

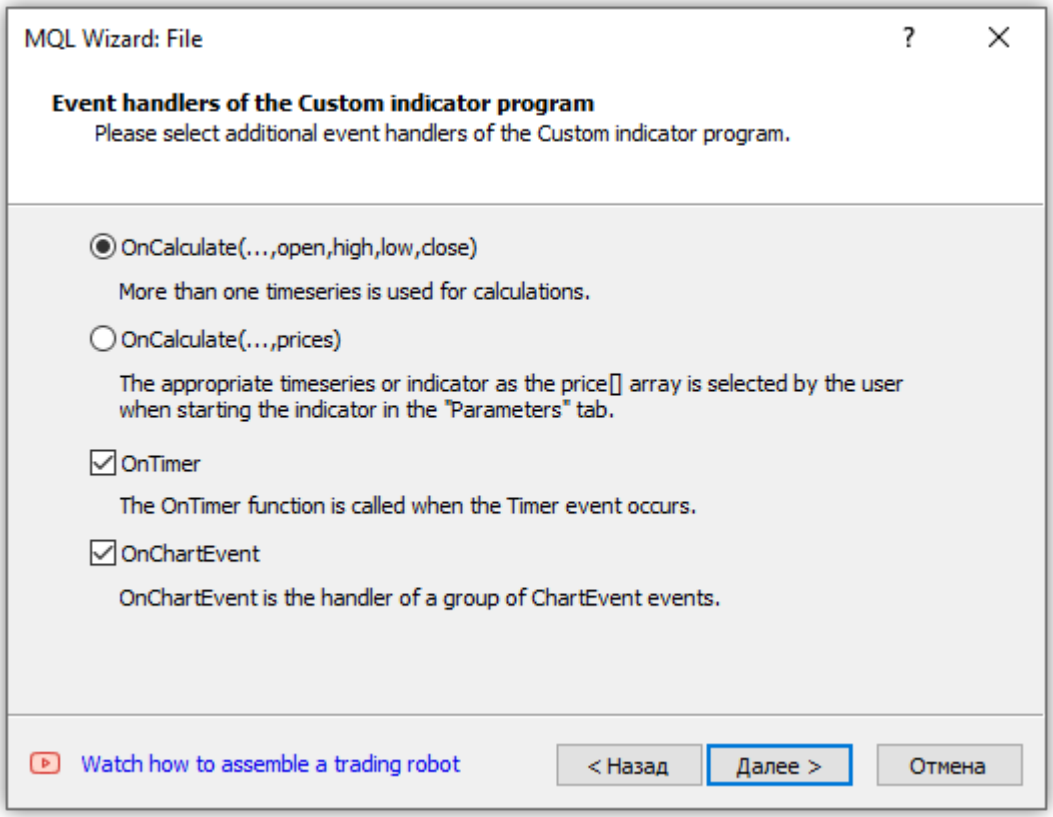
Внутри одного индикатора будем создавать два одинаковых индикатора. Один будет рассчитываться на данных текущего графика, а второй — на данных символа/периода графика, выбранных в настройках. Таким образом мы всегда сможем видеть линию скользящей средней на текущем графике и линию такой же скользящей средней, но построенную на другом периоде графика. Символ тоже можно выбрать от другого графика, но тогда линии не будут совпадать по уровню цен.

Создадим новый файл индикатора с именем TestMSTFMovingAverages.mq5:

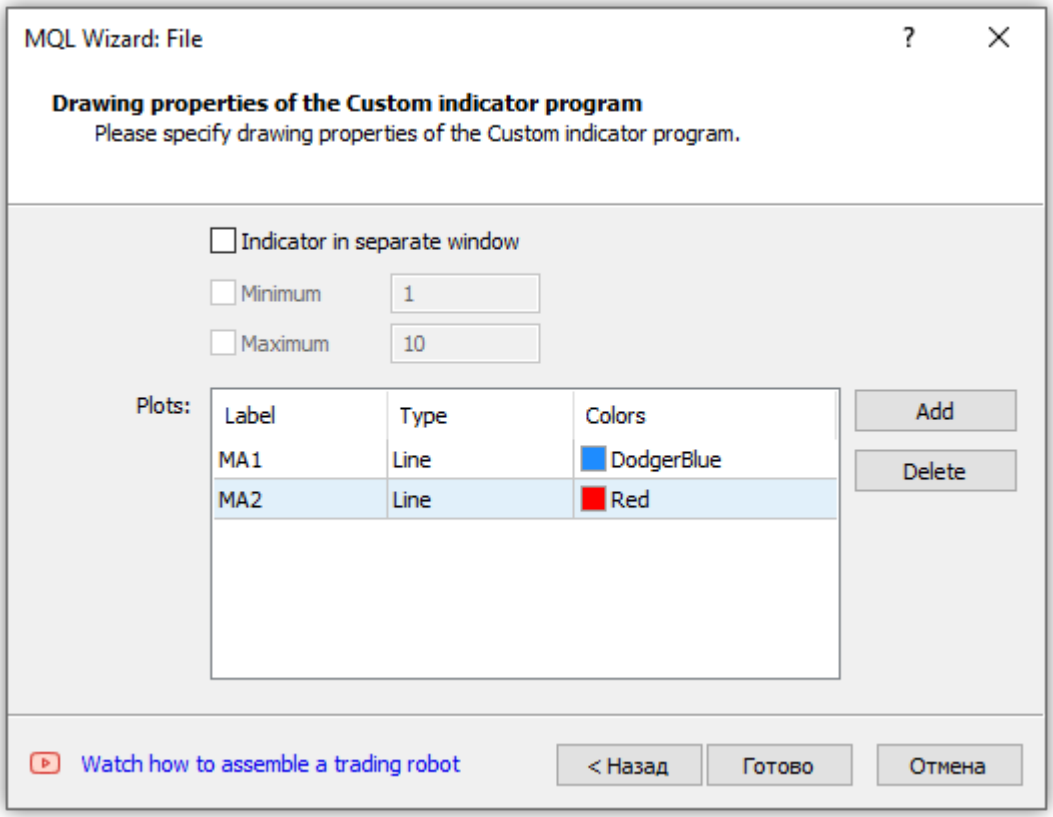




Отметим обработчики OnTimer и OnChartEvent:



Выберем два рисуемых буфера в виде линий в основном окне графика:



Имена буферов могут быть любыми — переименуем в коде. Нажимаем "Готово" и получаем шаблон индикатора:

```
//+------------------------------------------------------------------+
//|                                      TestMSTFMovingAverages.mq5 |
//|                        Copyright 2023, MetaQuotes Ltd.           |
//|                                             https://www.mql5.com |
//+------------------------------------------------------------------+
```

```
#property copyright "Copyright 2023, MetaQuotes Ltd."
#property link      "https://www.mql5.com"
#property version   "1.00"
#property indicator_chart_window
#property indicator_buffers 2
#property indicator_plots   2
//--- plot MA1
#property indicator_label1  "MA1"
#property indicator_type1   DRAW_LINE
#property indicator_color1  clrDodgerBlue
#property indicator_style1  STYLE_SOLID
#property indicator_width1  1
//--- plot MA2
#property indicator_label2  "MA2"
#property indicator_type2   DRAW_LINE
#property indicator_color2  clrRed
#property indicator_style2  STYLE_SOLID
#property indicator_width2  1
//--- indicator buffers
double         MA1Buffer[];
double         MA2Buffer[];
//+------------------------------------------------------------------+
//| Custom indicator initialization function                         |
//+------------------------------------------------------------------+
int OnInit()
  {
//--- indicator buffers mapping
   SetIndexBuffer(0,MA1Buffer,INDICATOR_DATA);
   SetIndexBuffer(1,MA2Buffer,INDICATOR_DATA);

//---
   return(INIT_SUCCEEDED);
  }
//+------------------------------------------------------------------+
//| Custom indicator iteration function                              |
//+------------------------------------------------------------------+
int OnCalculate(const int rates_total,
                const int prev_calculated,
                const datetime &time[],
                const double &open[],
                const double &high[],
                const double &low[],
                const double &close[],
                const long &tick_volume[],
                const long &volume[],
                const int &spread[])
  {
//---

//--- return value of prev_calculated for next call
   return(rates_total);
  }
//+------------------------------------------------------------------+
//| Timer function                                                   |
//+------------------------------------------------------------------+
void OnTimer()
  {
```

```
//---

  }
//+------------------------------------------------------------------+
//| ChartEvent function                                              |
//+------------------------------------------------------------------+
void OnChartEvent(const int id,
                  const long &lparam,
                  const double &dparam,
                  const string &sparam)
  {
//---

  }
```

В глобальной области добавим перечисление для выбора типа скользящей средней, подключим файл классов мультисимвольных мультипериодных индикаторов и файл с классом панели, и объявим входные переменные.
Переименуем буферы для более читабельного их названия и объявим глобальные переменные индикатора:

```
//+------------------------------------------------------------------+
//|                                        TestMSTFMovingAverages.mq5 |
//|                                Copyright 2023, MetaQuotes Ltd. |
//|                                             https://www.mql5.com |
//+------------------------------------------------------------------+
#property copyright "Copyright 2023, MetaQuotes Ltd."
#property link      "https://www.mql5.com"
#property version   "1.00"
#property indicator_chart_window
#property indicator_buffers 2
#property indicator_plots   2
//--- enums
enum ENUM_USED_MA
  {
   USED_MA_AMA    =  IND_AMA,    // Adaptive Moving Average
   USED_MA_DEMA   =  IND_DEMA,   // Double Exponential Moving Average
   USED_MA_FRAMA  =  IND_FRAMA,  // Fractal Adaptive Moving Average
   USED_MA_MA     =  IND_MA,     // Moving Average
   USED_MA_TEMA   =  IND_TEMA,   // Triple Exponential Moving Average
   USED_MA_VIDYA  =  IND_VIDYA,  // Variable Index Dynamic Average
  };
//--- plot MA1
#property indicator_label1  "MA1"
#property indicator_type1   DRAW_LINE
#property indicator_color1  clrDodgerBlue
#property indicator_style1  STYLE_SOLID
#property indicator_width1  1

//--- plot MA2
#property indicator_label2  "MA2"
#property indicator_type2   DRAW_LINE
#property indicator_color2  clrRed
#property indicator_style2  STYLE_SOLID
#property indicator_width2  1

//--- includes
```

```
#include <IndMSTF\IndMSTF.mqh>
#include <Dashboard\Dashboard.mqh>
//--- input parameters
input ENUM_USED_MA          InpIndicator    =  USED_MA_MA;       /* Used MA         */ //
input string                InpSymbol       =  NULL;             /* Symbol          */ //
input ENUM_TIMEFRAMES       InpTimeframe    =  PERIOD_CURRENT;   /* Timeframe       */ //
input ENUM_APPLIED_PRICE    InpPrice        =  PRICE_CLOSE;      /* Applied Price   */ //
input ENUM_MA_METHOD        InpMethod       =  MODE_SMA;         /* MA Method       */ //
input int                   InpShift        =  0;                /* MA Shift        */ //
input bool                  InpAsSeries     =  true;             /* As Series flag */ //

//--- indicator buffers
double          BufferMA1[];
double          BufferMA2[];
//--- global variables
int handle_ma1;
int handle_ma2;
CMSTFIndicators indicators;      // Экземпляр объекта коллекции индикаторов
//--- переменные для панели
CDashboard *panel=NULL;          // Указатель на объект панели
int         mouse_bar_index;     // Индекс бара, с которого берутся данные
```

Внутри обработчика OnInit() впишем создание таймера, назначим рисуемые буферы, создадим хэндлы выбранных индикаторов и создадим информационную панель:

```
//+------------------------------------------------------------------+
//| Custom indicator initialization function                         |
//+------------------------------------------------------------------+
int OnInit()
  {
//--- Устанавливаем таймер с периодичностью в 1 секунду
   EventSetTimer(1);
//--- Назначаем рисуемым буферам 0 и 1 массивы BufferMA1 и BufferMA2 соответственно
   SetIndexBuffer(0,BufferMA1,INDICATOR_DATA);
   SetIndexBuffer(1,BufferMA2,INDICATOR_DATA);
//--- sets indicator shift
   //PlotIndexSetInteger(0,PLOT_SHIFT,InpShift);   // аналог в стр. 116
   //PlotIndexSetInteger(1,PLOT_SHIFT,InpShift);   // аналог в стр. 117
//--- Устанавливаем флаги серийности массивам буферов индикатора (для теста, чтобы видн
   ArraySetAsSeries(BufferMA1,InpAsSeries);
   ArraySetAsSeries(BufferMA2,InpAsSeries);

//--- Для разных индикаторов ширина панели (width) будет индивидуальной (из-за количест
   int width=0;
//--- В зависимости от выбранного в настройках индикатора создаём два индикатора одного
//--- Первый рассчитывается на текущих символе/периоде графика, а второй - на тех, что
   switch(InpIndicator)
     {
      case USED_MA_AMA     :
         handle_ma1=indicators.AddNewAMA(NULL,PERIOD_CURRENT,9,2,30,InpShift);
         handle_ma2=indicators.AddNewAMA(InpSymbol,InpTimeframe,9,2,30,InpShift);
         width=269;
        break;
      case USED_MA_DEMA    :
         handle_ma1=indicators.AddNewDEMA(NULL,PERIOD_CURRENT,14,InpShift,InpPrice);
         handle_ma2=indicators.AddNewDEMA(InpSymbol,InpTimeframe,14,InpShift,InpPrice);
         width=255;
```

```
         break;
      case USED_MA_FRAMA   :
         handle_ma1=indicators.AddNewFrAMA(NULL,PERIOD_CURRENT,14,InpShift,InpPrice);
         handle_ma2=indicators.AddNewFrAMA(InpSymbol,InpTimeframe,14,InpShift,InpPrice)
         width=259;
        break;
      case USED_MA_TEMA    :
         handle_ma1=indicators.AddNewTEMA(NULL,PERIOD_CURRENT,14,InpShift,InpPrice);
         handle_ma2=indicators.AddNewTEMA(InpSymbol,InpTimeframe,14,InpShift,InpPrice);
         width=253;
        break;
      case USED_MA_VIDYA   :
         handle_ma1=indicators.AddNewVIDyA(NULL,PERIOD_CURRENT,9,12,InpShift,InpPrice);
         handle_ma2=indicators.AddNewVIDyA(InpSymbol,InpTimeframe,9,12,InpShift,InpPric
         width=267;
        break;
      default:
         handle_ma1=indicators.AddNewMA(NULL,PERIOD_CURRENT,10,InpShift,InpMethod,InpPr
         handle_ma2=indicators.AddNewMA(InpSymbol,InpTimeframe,10,InpShift,InpMethod,In
         width=231;
        break;
     }
//--- Если не удалось создать хэндлы индикаторов - возвращаем ошибку инициализации
   if(handle_ma1==INVALID_HANDLE || handle_ma2==INVALID_HANDLE)
      return INIT_FAILED;
//--- Устанавливаем описания линий индикатора из описаний буферов расчётной части создан
   indicators.SetPlotLabelFromBuffer(0,handle_ma1,0);
   indicators.SetPlotLabelFromBuffer(1,handle_ma2,0);
//--- Устанавливаем "пустые" значения для буферов расчётной части созданных индикаторов
   indicators.SetBufferInitValue(handle_ma1,0,EMPTY_VALUE);
   indicators.SetBufferInitValue(handle_ma2,0,EMPTY_VALUE);
//--- Устанавливаем смещения линиям индикатора
   indicators.SetPlotShift(0,InpShift);
   indicators.SetPlotShift(1,InpShift);

//--- Панель
//--- Создаём панель
   panel=new CDashboard(1,20,20,width,264);
   if(panel==NULL)
     {
      Print("Error. Failed to create panel object");
      return INIT_FAILED;
     }
//--- Устанавливаем параметры шрифта
   panel.SetFontParams("Calibri",9);
//--- Отображаем панель с текстом в заголовке "Символ, Описание таймфрейма"
   panel.View(Symbol()+", "+StringSubstr(EnumToString(Period()),7));

//--- Создаём таблицу с идентификатором 0 для отображения в ней данных бара
   panel.CreateNewTable(0);
//--- Рисуем таблицу с идентификатором 0 на фоне панели
   panel.DrawGrid(0,2,20,6,2,18,width/2-2);

//--- Создаём таблицу с идентификатором 1 для отображения в ней данных индикатора 1
   panel.CreateNewTable(1);
//--- Получаем координату Y2 таблицы с идентификатором 0 и
//--- устанавливаем координату Y1 для таблицы с идентификатором 1
```

```
   int y1=panel.TableY2(0)+22;
//--- Рисуем таблицу с идентификатором 1 на фоне панели
   panel.DrawGrid(1,2,y1,2,2,18,width/2-2);

//--- Создаём таблицу с идентификатором 2 для отображения в ней данных индикатора 2
   panel.CreateNewTable(2);
//--- Получаем координату Y2 таблицы с идентификатором 1 и
//--- устанавливаем координату Y1 для таблицы с идентификатором 2
   int y2=panel.TableY2(1)+3;
//--- Рисуем таблицу с идентификатором 2 на фоне панели
   panel.DrawGrid(2,2,y2,3,2,18,width/2-2);

//--- Инициализируем переменную с индексом бара указателя мышки
   mouse_bar_index=0;
//--- Выводим на панель данные текущего бара
   DrawData(mouse_bar_index,TimeCurrent());

//--- Успешная инициализация
   return(INIT_SUCCEEDED);
  }
```

Добавим обработчик OnDeinit():

```
//+------------------------------------------------------------------+
//| Custom indicator deinitialization function                       |
//+------------------------------------------------------------------+
void OnDeinit(const int reason)
  {
//--- Уничтожаем таймер
   EventKillTimer();
//--- Если объект панели существует - удаляем его
   if(panel!=NULL)
      delete panel;
//--- Стираем все комментарии
   Comment("");
  }
```

В обработчике OnCalculate() вызовем расчёт всех мультисимвольных мультипериодных индикаторов. При неудачном расчёте нужно выйти из обработчика с возвратом нуля — чтобы на следующем тике пересчитать индикаторы заново.
После успешного расчёта данные в массивах-буферах объектов-индикаторов уже есть — их можно уже вывести в информационную панель. После вывода данных на панель, выведем данные из расчитанных буферов в рисуемые буферы индикатора:

```
//+------------------------------------------------------------------+
//| Custom indicator iteration function                              |
//+------------------------------------------------------------------+
int OnCalculate(const int rates_total,
                const int prev_calculated,
                const datetime &time[],
                const double &open[],
                const double &high[],
                const double &low[],
                const double &close[],
                const long &tick_volume[],
                const long &volume[],
```

```
                const int &spread[])
  {
//--- Количество баров для расчёта
   int limit=rates_total-prev_calculated;
//--- Если limit > 1, значит это первый расчёт, либо изменение в истории
   if(limit>1)
     {
      //--- указываем для просчёта всю доступную историю
      limit=rates_total-1;
      /*
      // Если в индикаторе есть какие-либо буферы, в которых отображены другие расчёты
      // то здесь их нужно инициализировать "пустым" значением, установленным для этих
      */
     }
//--- Рассчитываем все созданные мультисимвольные мультипериодные индикаторы
   if(!indicators.Calculate())
      return 0;

//--- Выводим на панель данные бара под курсором (либо текущий бар, если курсор за пред
   DrawData(mouse_bar_index,time[mouse_bar_index]);

//--- Из буферов рассчитанных индикаторов выводим данные в индикаторные буферы
   if(!indicators.DataToBuffer(NULL,PERIOD_CURRENT,handle_ma1,0,limit,BufferMA1))
      return 0;
   if(!indicators.DataToBuffer(NULL,PERIOD_CURRENT,handle_ma2,0,limit,BufferMA2))
      return 0;

//--- return value of prev_calculated for next call
   return(rates_total);
  }
```

В таймере индикатора вызываем таймер объекта-коллекции индикаторов:

```
//+------------------------------------------------------------------+
//| Timer function                                                   |
//+------------------------------------------------------------------+
void OnTimer()
  {
//--- Вызываем таймер коллекции индикаторов
   indicators.OnTimer();
  }
```

В обработчике событий графика OnChartEvent() вызываем обработчик событий объекта-панели и обрабатываем перемещение курсора для определения бара, над которым он находится:

```
//+------------------------------------------------------------------+
//| ChartEvent function                                              |
//+------------------------------------------------------------------+
void OnChartEvent(const int id,
                  const long &lparam,
                  const double &dparam,
                  const string &sparam)
  {
//--- Работа с панелью
//--- Вызываем обработчик событий панели
   panel.OnChartEvent(id,lparam,dparam,sparam);
```

```
//--- Если курсор перемещается или щелчок по графику
   if(id==CHARTEVENT_MOUSE_MOVE || id==CHARTEVENT_CLICK)
     {
      //--- Объявляем переменные для записи в них координат времени и цены
      datetime time=0;
      double price=0;
      int wnd=0;
      //--- Если координаты курсора преобразованы в дату и время
      if(ChartXYToTimePrice(ChartID(),(int)lparam,(int)dparam,wnd,time,price))
        {
         //--- записываем индекс бара, где расположен курсор в глобальную переменную
         mouse_bar_index=iBarShift(Symbol(),PERIOD_CURRENT,time);
         //--- Выводим данные бара под курсором на панель
         DrawData(mouse_bar_index,time);
        }
     }

//--- Если получили пользовательское событие - выводим об этом сообщение в журнал
   if(id>CHARTEVENT_CUSTOM)
     {
      //--- Здесь может быть обработка щелчка по кнопке закрытия на панели
      PrintFormat("%s: Event id=%ld, object id (lparam): %lu, event message (sparam): %
     }
  }
```

Функция, выводящая данные мульти- индикаторов на панель:

```
//+------------------------------------------------------------------+
//| Выводит данные с указанного индекса таймсерии на панель          |
//+------------------------------------------------------------------+
void DrawData(const int index,const datetime time)
  {
//--- Объявляем переменные для получения в них данных
   MqlRates rates[1];

//--- Если данные бара по указанному индексу получить не удалось - уходим
   if(CopyRates(Symbol(),PERIOD_CURRENT,index,1,rates)!=1)
      return;

//--- Устанавливаем параметры шрифта для заголовков данных бара и индикатора
   int  size=0;
   uint flags=0;
   uint angle=0;
   string name=panel.FontParams(size,flags,angle);
   panel.SetFontParams(name,9,FW_BOLD);
   panel.DrawText("Bar data ["+(string)index+"]",3,panel.TableY1(0)-16,clrMaroon,panel.
   panel.DrawText("Indicators data ["+(string)index+"]",3,panel.TableY1(1)-16,clrGreen,
//--- Устанавливаем параметры шрифта для данных бара и индикатора
   panel.SetFontParams(name,9);

//--- Выводим на панель данные указанного бара в таблицу 0
   panel.DrawText("Date",  panel.CellX(0,0,0)+2, panel.CellY(0,0,0)+2); panel.DrawText(
   panel.DrawText("Time",  panel.CellX(0,1,0)+2, panel.CellY(0,1,0)+2); panel.DrawText(
   panel.DrawText("Open",  panel.CellX(0,2,0)+2, panel.CellY(0,2,0)+2); panel.DrawText(
   panel.DrawText("High",  panel.CellX(0,3,0)+2, panel.CellY(0,3,0)+2); panel.DrawText(
   panel.DrawText("Low",   panel.CellX(0,4,0)+2, panel.CellY(0,4,0)+2); panel.DrawText(
   panel.DrawText("Close", panel.CellX(0,5,0)+2, panel.CellY(0,5,0)+2); panel.DrawText(
```

```
//--- Выводим в таблицу 1 данные индикатора 1 с указанного бара в таблицу 1
   panel.DrawText(indicators.Title(handle_ma1), panel.CellX(1,0,0)+2, panel.CellY(1,0,0
   double value1=indicators.GetData(handle_ma1,0,index);
   string value_str1=(value1!=EMPTY_VALUE ? DoubleToString(value1,indicators.Digits(har
   panel.DrawText(value_str1,panel.CellX(1,0,1)+2,panel.CellY(1,0,1)+2,clrNONE,110);

//--- Выводим описание состояния линии индикатора 1
   panel.DrawText("Line state", panel.CellX(1,1,0)+2, panel.CellY(1,1,0)+2);
   ENUM_LINE_STATE state1=indicators.BufferLineState(Symbol(),PERIOD_CURRENT,handle_ma1
   panel.DrawText(BufferLineStateDescription(state1),panel.CellX(1,1,1)+2,panel.CellY(1

//--- Выводим в таблицу 2 данные индикатора 2 с указанного бара в таблицу 2
   panel.DrawText(indicators.Title(handle_ma2), panel.CellX(2,0,0)+2, panel.CellY(2,0,0
   double value2=indicators.GetDataTo(Symbol(),PERIOD_CURRENT,handle_ma2,0,index);
   string value_str2=(value2!=EMPTY_VALUE ? DoubleToString(value2,indicators.Digits(har
   panel.DrawText(value_str2,panel.CellX(2,0,1)+2,panel.CellY(2,0,1)+2,clrNONE,110);

//--- Выводим описание состояния линии индикатора 2
   panel.DrawText("Line state", panel.CellX(2,1,0)+2, panel.CellY(2,1,0)+2);
   ENUM_LINE_STATE state2=indicators.BufferLineState(Symbol(),PERIOD_CURRENT,handle_ma2
   panel.DrawText(BufferLineStateDescription(state2),panel.CellX(2,1,1)+2,panel.CellY(2

//--- Выводим описание соотношения линии индикатора 1 относительно линии индикатора 2
   double value21=indicators.GetDataTo(Symbol(),PERIOD_CURRENT,handle_ma2,0,index+1);
   ENUM_LINE_STATE stateR=indicators.BufferLineStateRelative(Symbol(),PERIOD_CURRENT,ha
   string ma1=indicators.Name(handle_ma1);
   string ma2=indicators.Name(handle_ma2);
   string state_relative=
     (
      stateR==LINE_STATE_ABOVE      ? StringFormat("%s1 > %s2",ma1,ma2)    :
      stateR==LINE_STATE_BELOW      ? StringFormat("%s1 < %s2",ma1,ma2)           :
      stateR==LINE_STATE_CROSS_DOWN ? "Top-down crossing"    :
      stateR==LINE_STATE_CROSS_UP   ? "Bottom-up crossing"   :
      BufferLineStateDescription(stateR)
     );
   panel.DrawText(StringFormat("%s1 vs %s2",ma1,ma2), panel.CellX(2,2,0)+2, panel.CellY
   panel.DrawText(state_relative, panel.CellX(2,2,1)+2, panel.CellY(2,2,1)+2,clrNONE,11

//--- Перерисовываем график для немедленного отображения всех изменений на панели
   ChartRedraw(ChartID());
  }
```

Как видим, для расчёта мультисимвольных мультипериодных индикаторов достаточно вызвать метод Calculate() класса-коллекции индикаторов. После успешного расчёта все данные уже доступны. В советниках их можно уже получать и обрабатывать. В индикаторах после успешного расчёта данные можно вывести на график в виде линий рисуемых буферов при помощи метода DataToBuffer() класса-коллекции индикаторов. И это всё, что нужно для расчёта и отображения мульти- индикаторов на графике.

После компиляции тестового индикатора запустим его на графике с периодом M1, в настройках выберем текущий символ и период расчёта индикатора M5. В этом случае будут созданы два выбранных в настройках индикатора скользящих средних. Один будет рассчитан на данных текущего графика, а второй — на данных пятиминутного периода графика. Переключая таймфрейм графика можно будет видеть как на M1 рисуются две линии: одна будет соответствовать скользящей средней, рассчитанной на M1, а вторая — скользящей средней, рассчитанной на M5. Если переключить график на M5, то будет

создан только один индикатор, так как второй будет идентичен первому, и он создан не будет. Если переключить график на M15, то один индикатор будет рассчитан на M15, а второй — на M5, и он тоже будет отображён на графике.



Как видим, заявленный функционал работает при отображении в главном окне графика индикаторов с одним буфером.

Все файлы всех классов и тестового индикатора можно посмотреть в прилагаемых к статье файлах.

Заключение

Сегодня мы сделали функционал для быстрого создания мультисимвольных мультипериодных индикаторов и получения их данных в индикаторах с отрисовкой просчитанных индикаторов в главном окне графика. В следующих статьях сделаем шаблоны для создания других мультисимвольных мультипериодных стандартных индикаторов, рисующих свои данные в главном окне графика, и не только с одним рисуемым буфером. В итоге у нас получится удобный инструмент для быстрого превращения любых индикаторов в их мультисимвольные мультипериодные версии. И, скорее всего, классы мульти- индикаторов далее будут дорабатываться при создании других стандартных и пользовательских индикаторов.

Полный исходный код проекта со всеми файлами, описанными в статье, доступен в репозитории.

Следующая статья >>

Artyom Trishkin

Россия 87827

Пишу скрипты, индикаторы, советники на mql5

По всем вопросам, пожалуйста обращайтесь в личных сообщениях.

For all questions, please contact personal messages.
==========

Другие статьи автора

- Код, слёзы и Algo Forge
- Индикатор CandleCode: Формализация свечных моделей в MQL5
- Забытая классика объёма: индикатор "Finite Volume Elements" для современных рынков
- Математика волатильности: Почему индикатор GRI достоин возвращения в ваш торговый терминал
- Таблицы в парадигме MVC на MQL5: Таблица корреляции символов
- Автоматизация запуска терминала для выполнения сервисных задач
- Таблицы в парадигме MVC на MQL5: настраиваемые и сортируемые столбцы таблицы

Подпишитесь и узнавайте о
популярных статьях и кодах